TP START Spring Boot

Débuter avec le Framework Spring Boot



Plan Général du TP

Objectifs

Objectifs du TP

- **Durant ce TP vous réaliserez les actions suivantes :**

  - Présentation générale de **Spring Boot**
  - Concepts fondamentaux associés aux Frameworks
  - Initialiser un projet **Spring Boot** avec **Spring Initializr**
  - Ouvrir et exécuter le projet initialisé pour vérifier la satisfaction de la configuration d'**IntelliJ** et de notre projet (*proxy* et *port* du serveur Web)

Objectifs du TP

- **Durant ce TP vous réaliserez les actions suivantes :**

  - Appréhender la structure générale d'un projet **Spring Boot**

  - Création d'un premier Contrôleur :
    - Faire afficher le fameux **« Hello World ! »** dans notre navigateur
    - Introduire la notion de **endpoint** (point terminal/URI)
    - Voir ou revoir le concept de corps d'une requête **HTTP**

  - Manipuler des éléments de base de la couche **Vue**.

Présentation Générale de Spring Boot

Pourquoi utiliser un Framework ?

Pourquoi le choix de Spring Boot ?

Pourquoi utiliser un Framework ?

- **Etat de lieux :**

  - Vous avez déjà accumulé de beaucoup de compétences et de connaissances vous permettant de réaliser de façon satisfaisante des solutions :

    - Réaliser de programmes/script s'exécutant en mode console et cela dans différents langages (**Python**, **Kotlin**, etc).

    - Réaliser des sites Web statiques (**HTML** et **CSS**).

    - Ajouter du comportement dynamique côté client avec **JavaScript** (le 2e semestre sera articulé sur la réalisation d'un projet orienté Front-end en **JavaScript**/**TypeScript** avec le Framework **Angular**.

    - Travailler avec des bases de données relationnelles **SQL**.

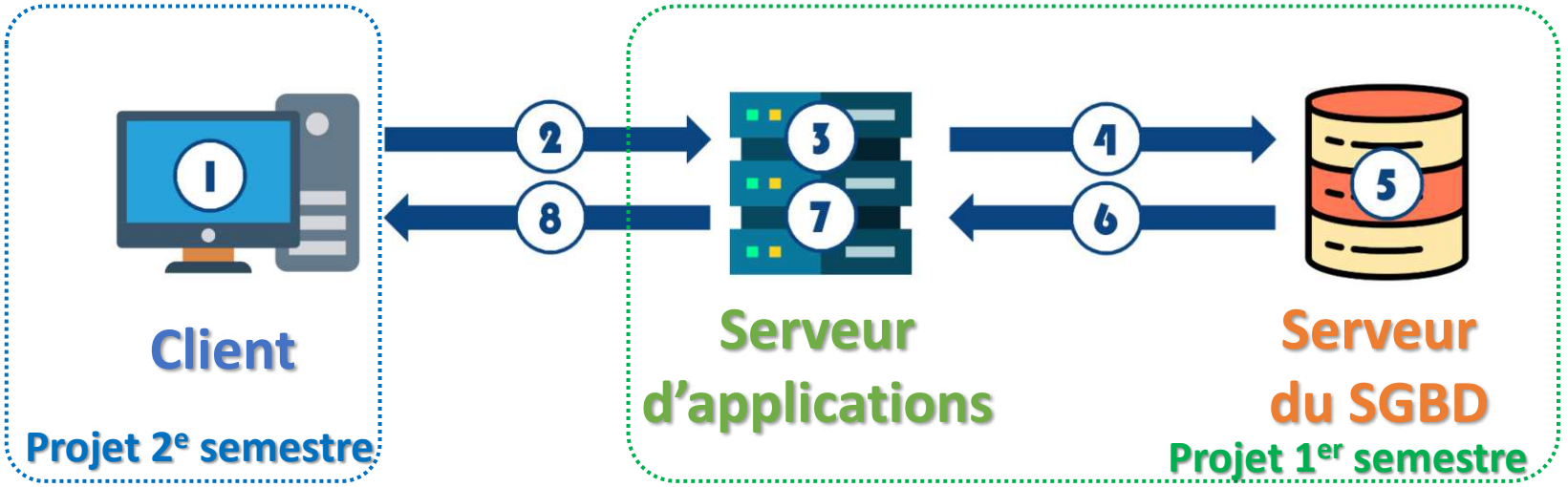
Pourquoi utiliser un Framework ?

- **Des problèmes à venir… sans Framework :**
  - Nous allons maintenant être venu à réaliser des applications plus complexes qui vont mettre en œuvre :
    - **Différentes technologies :**
      - **Côté client** : HTML/CSS, JavaScript…
      - **Côté serveur d'application + serveur SGBD** : Application métier proposant un nombre plus ou moins conséquent de composants/services

Pourquoi utiliser un Framework ?

- **Des problèmes à venir… sans Framework**

  - Il est de coutume d'avoir une première approche/point de vue avec le modèle d'architecture des 3 tiers :
    - **Client** (navigateur sur PC/mobile/tablette; application mobile; application de bureau…)
    - **Serveur d'applications**
    - **Serveur du SGBD**



Pourquoi utiliser un Framework ?

- **Des problèmes à venir… sans Framework**

  - Pour l'instant nous nous concentrons sur les 2 derniers tiers (serveur applications & serveur SGBD).

  - Mais il est très difficile de réaliser « from scratch » des applications :

    - **Fiables :** Bogues limités + rapidité des correctifs

    - **Sécurisées** : Mise en œuvre de bonnes pratiques éprouvées en termes de sécurité

    - **Maintenables & évolutifs :** Pouvoir faire évoluer ses applications, remplacer une partie, voire plusieurs en minimisant les modifications sur les composants inchangés.

Pourquoi utiliser un Framework ?

- **S'appuyer sur les acquis d'une communauté informatique**
  - Les Framework sont produits par des sociétés et/ou des communautés open source
    - Par nature un Framework bénéficie des savoir-faire et expériences d'acteurs
    - Ces acteurs ont procédé à des choix : architecture, bibliothèques par exemple
    - Ont su développer la communauté initiale, permettant un développement pérenne du Framework tout en ayant une meilleure réactivité pour combler des failles, corriger des bogues et proposer de nouvelles fonctionnalités.

Pourquoi utiliser un Framework ?

- **Que des avantages ?**
  - Non, il y a des compromis :
    - Choisir un Framework parmi l'offre disponible n'est pas évident selon la situation.

Langage Java :

- Spring MVC
- Spring Boot



Langage Python :

- Django



Langage Ruby :

- Ruby on Rails



Langage JavaScript :

- Express.js



Langage PHP :

- Laravel



- Symfony



Pourquoi utiliser un Framework ?

- **Que des avantages ?**
  - Non, il y a des compromis :
    - Il y a une courbe d'apprentissage qui peut être longue.
    - Il est difficile de migrer d'un Framework vers un autre pour une application existante.

Pourquoi le choix de Spring Boot ?

- **Une démarche raisonnée :**

  - Le choix du Framework **Spring Boot** a été motivé par la prise en compte de plusieurs critères :

    - Le langage : **Java** & **Kotlin**
    - L'architecture implémentée : **M**odèle – **V**ue – **C**ontrôleur
    - La bonne collaboration avec l'IDE **IntelliJ**
    - L'adoption de **Spring** par de nombreux acteurs

Concepts fondamentaux

Concepts liés aux Frameworks

Concepts liés aux Frameworks

- **Mais d'abord c'est quoi un Framework ?**

  - Framework que nous pouvons traduire par *« cadre de travail »* ou *« cadre applicatif »* est un ensemble cohérent de composants logiciels préconçus, qui fournissent un squelette standardisé pour nos applications.

Concepts liés aux Frameworks

- **Le concept fondateur : L'Inversion de Contrôle (*IoC : Inversion of Control*)**

  - Si vous deviez partir de zéro (*from scratch*) vous devriez développer le programme principal (main) de votre application.

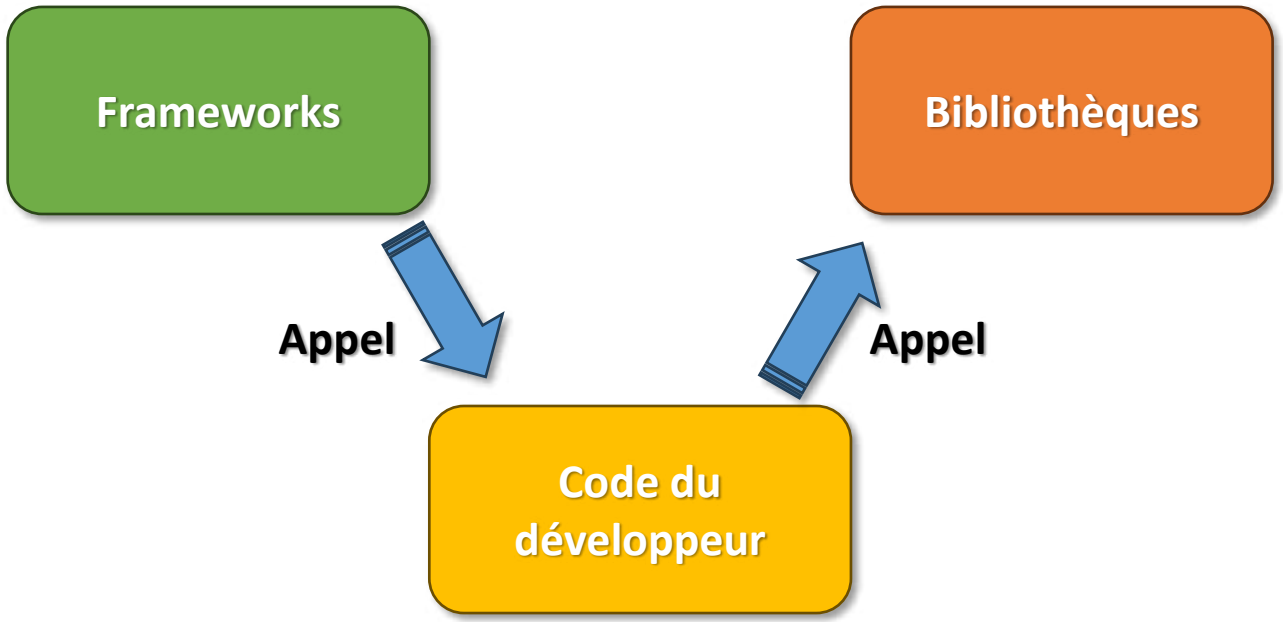
  - Ce programme à la délicate mission de chef d'orchestre de votre application :
    - D'instancier différents objets
    - De mettre en relation les objets par des associations (concept de dépendance)
    - Le tout de façon coordonnée.

Concepts liés aux Frameworks

- **Le concept fondateur : L'Inversion de Contrôle (*IoC : Inversion of Control*)**
  - Dans le cadre d'un développement avec un Framework c'est ce dernier qui mettra en œuvre toute cette délicate mécanique.
  - Le développeur se concentre sur l'implémentation de composants qui seront ensuite pris en charge par le Framework.
  - Cependant le développeur garde la main sur les bibliothèques tierces sur lesquelles s'appuie son code métier.

Concepts liés aux Frameworks

- **Le concept fondateur : L'Inversion de Contrôle (*IoC : Inversion of Control*)**
  - C'est cela que nous appelons l'Inversion de Contrôle :

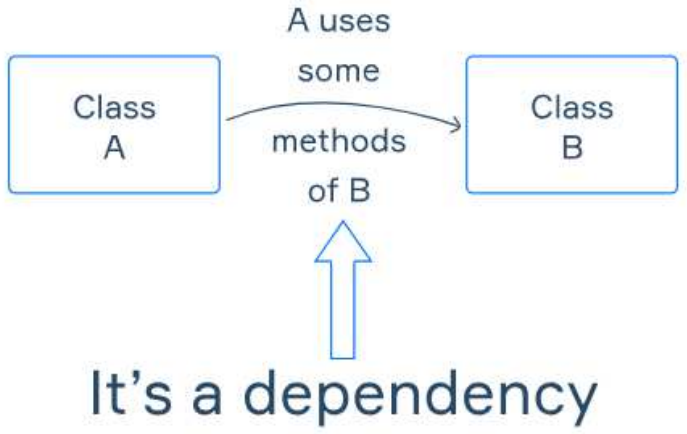


Concepts liés aux Frameworks

- **Le concept de dépendance**

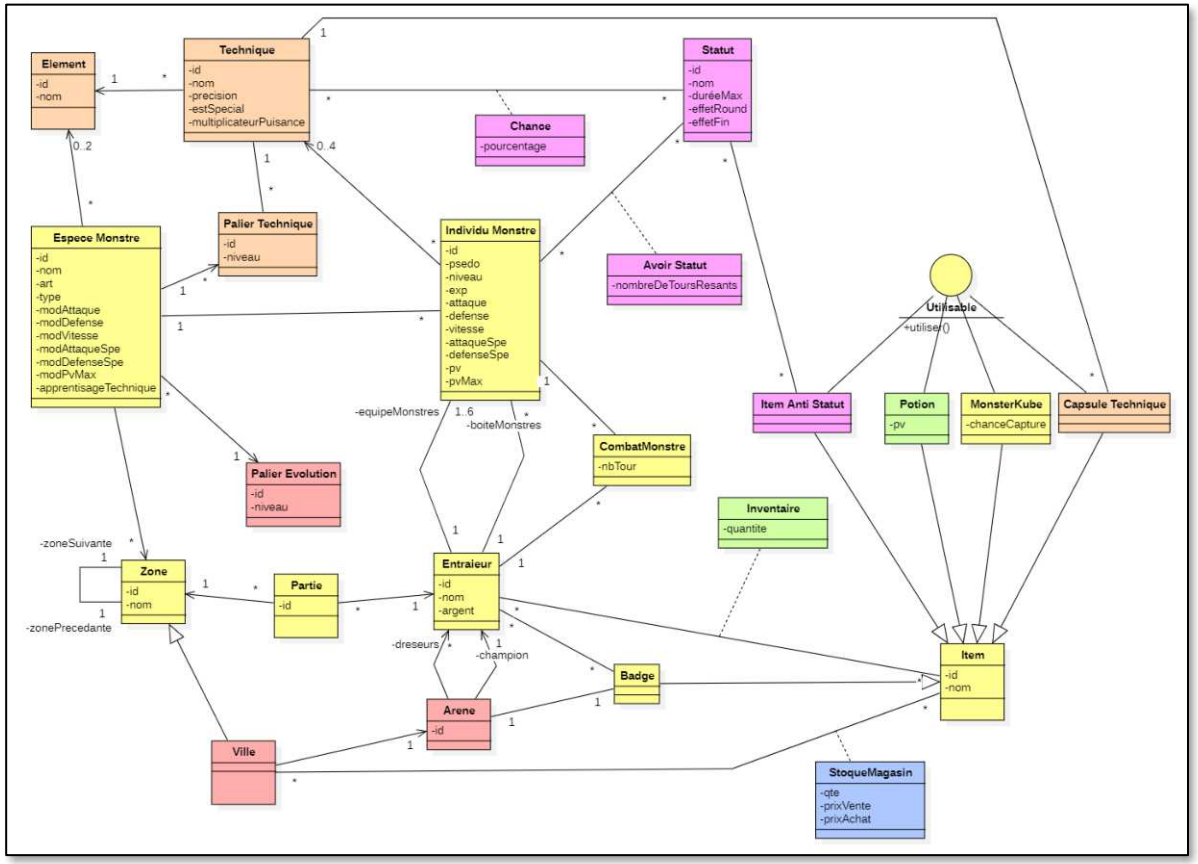
  - Le concept de dépendance, vous n'avez cessé de le manipuler depuis le début de l'année, mais nous ne l'avions pas explicitement donné de nom.

  - En effet à chaque fois qu'une classe utilise une autre classe, vous êtes en présence d'une dépendance :



Concepts liés aux Frameworks

- **Le concept de dépendance**
  - Durant les 3 premiers sprints du projet **KotlinMonsters**, vous aviez pu constater qu'il était plus pratique de coder certaines classes avant d'autres.



Concepts liés aux Frameworks

- **Le concept de faible couplage d'injection de dépendances**

  - Le principe de faible de couplage est de choisir et d'implémenter des relations entre les classes de notre application qui facilite le remplacement d'une classe sans nous retrouver dans l'obligation de modifier le code des classes dépendantes.

  - De facto, en mettant en œuvre des relations d'associations, nous avons mis en place une stratégie permettant d'avoir des relations de faible couplage entre nos classes.

Concepts liés aux Frameworks

- **Le concept de faible couplage d'injection de dépendances**

  - L'injection de dépendance est l'implémentation adoptée pour nos classes dépendantes afin qu'elle puisse accéder aux autres classes (dont elles dépendent) via leurs références.

Concepts liés aux Frameworks

- **Le concept de faible couplage d'injection de dépendances**

  - On distingue généralement **4 modes d'injections** de dépendances :

Injection par le constructeur : Les dépendances sont fournies à l'objet au moment de sa création via son constructeur.

Injection par setter : L'objet est d'abord créé, puis ses dépendances lui sont fournies via des méthodes setter.

Injection par propriété : Les propriétés ou attributs publics de l'objet sont utilisés pour lui fournir ses dépendances après sa création.

Injection par méthode : Les dépendances sont fournies via des méthodes dédiées.

Concepts liés aux Frameworks

- **Injection des dépendances par les annotations :**

  - Les Frameworks comme **Spring Boot** utilisent également une autre manière d'injecter des dépendances, en utilisant des **annotations**.

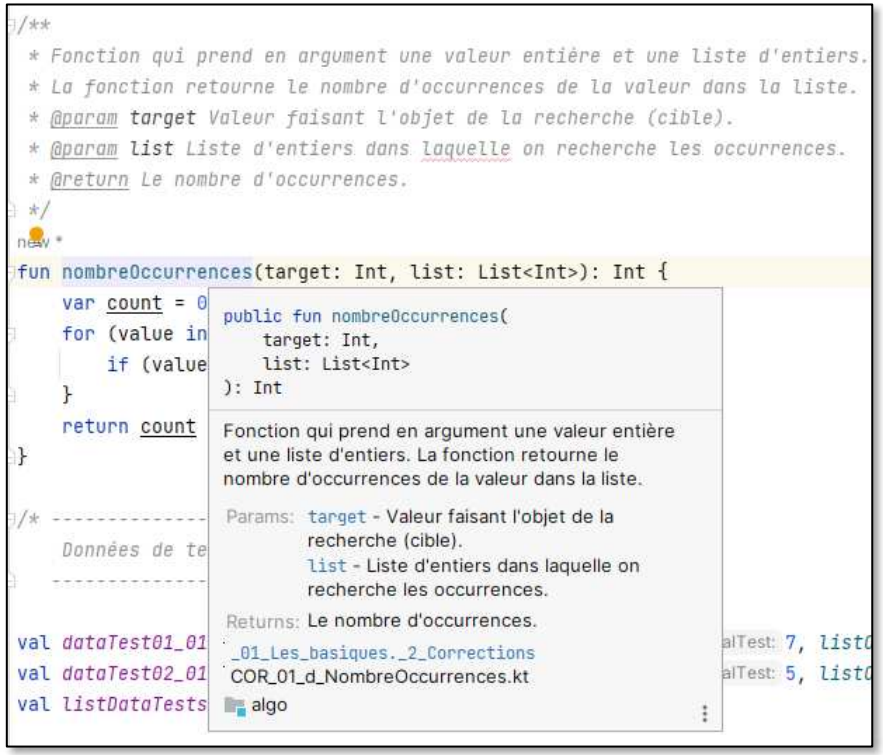
  - Les **annotations** sont reconnaissables car elles commencent par le symbole **@**

  - Plus généralement, une annotation est une métadonnée, c'est-à-dire une information (ou donnée) non destinée au compilateur du langage mais pour d'autres outils (builder d'application du Framework, IDE, etc.).

Concepts liés aux Frameworks

- **Injection des dépendances par les annotations :**

  - Jusqu'à présent, on utilisait les annotations pour produire de la documentation.



Concepts liés aux Frameworks

- **Injection des dépendances par les annotations :**

  - **Spring Boot** dispose d'un lot d'annotations permettant de prendre la main sur notre code (principe ***IoC***).

  - Nous serons menés à agrémenter nos classes avec des annotations de type :

    - **@Component** : Pour tout simplement annoter une classe métier de notre application

    - **@Repository** : Pour annoter les **DAO** (voirTP DAO & JDBC).
    - Etc.

Concepts liés aux Frameworks

- **Le modèle d'architecture MVC :**

  - Afin de structurer l'organisation du code de l'application et de définir des couches de responsabilité qui facilite l'évolution de l'application, chaque Framework adopte un modèle d'architecture.

  - **Spring Boot** ne déroge pas à la règle et adopte le modèle d'architecture le plus répandu, à savoir le modèle **MVC** : pour **M**odèle – **V**ue – **C**ontrôleur

Concepts liés aux Frameworks

- **Le modèle d'architecture MVC :**

  - En termes de responsabilités, nous avons la répartition suivante :

    - **Modèle :** Accueil la **logique métier**, c'est généralement la partie la moins soumise aux évolutions. Elle assure également la liaison avec la base de données.

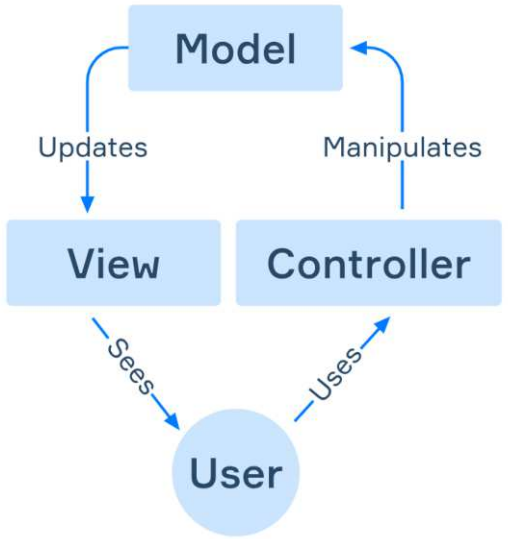
    - **Vue :** Assure la **présentation** des données à l'utilisateur et gère les interactions avec ce dernier.

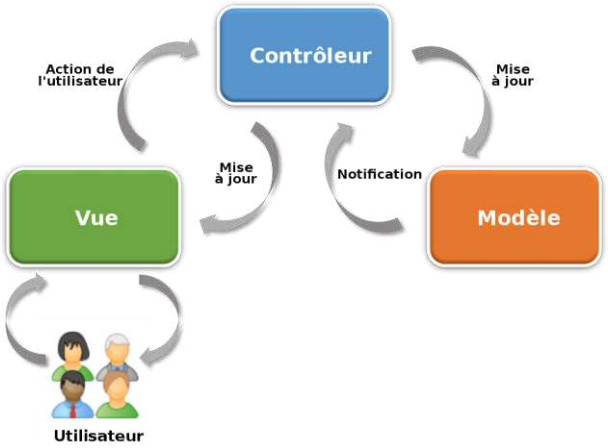
    - **Contrôleur :** Gère les requêtes émises par le client, assure la liaison avec la couche modèle et renvoie les réponses soit sous le format d'une vue (site web) ou de données (au format **JSON** par exemple) dans le cadre d'une **API**.

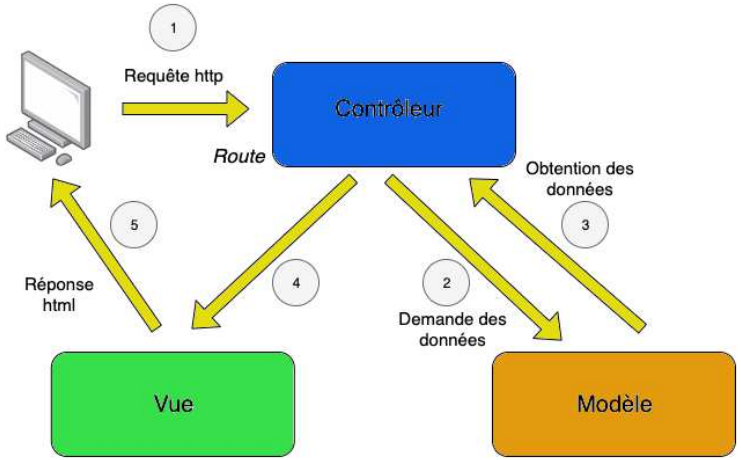
Concepts liés aux Frameworks

- **Le modèle d'architecture MVC :**

  - Nous pouvons trouver des variantes sur les liaisons des différentes couches, mais cela dépend en pratique des choix d'implémentation du Framework par son éditeur.







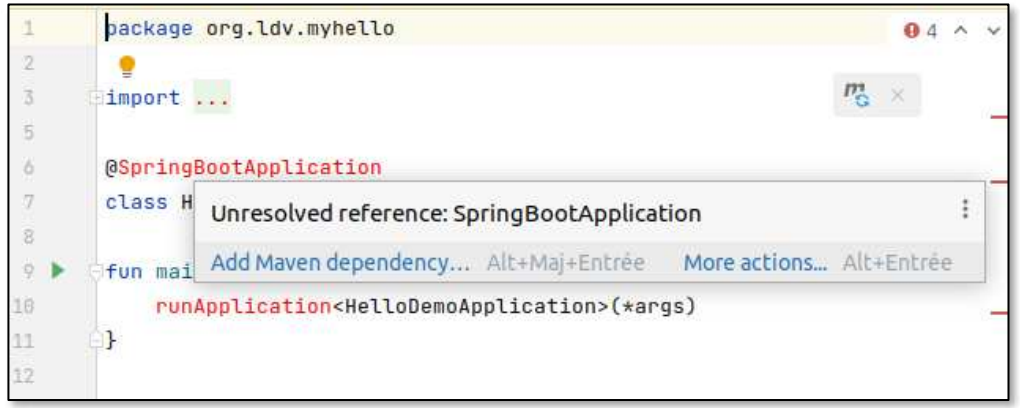
Configuration du Proxy pour Gradle

Spring Boot & IntelliJ

Spring Boot & IntelliJ

- **Cas du travail avec un proxy :**
  - La connexion Internet du lycée passe par un serveur proxy.
  - Il en découle que **Gradle** ne pourra accéder à ses dépôts pour télécharger les dépendances (bibliothèques tierces).
  - Vous aurez alors des alertes :





Spring Boot & IntelliJ

- **Cas du travail avec un proxy :**
  - Si vous avez appliqué le script **Python** de configuration sur votre session, ce dernier à normalement déjà réalisé le nécessaire pour renseigner le proxy de votre poste à la fois pour **Maven** et **Gradle**.




```
# Appliquer configurations :
# --------------------------

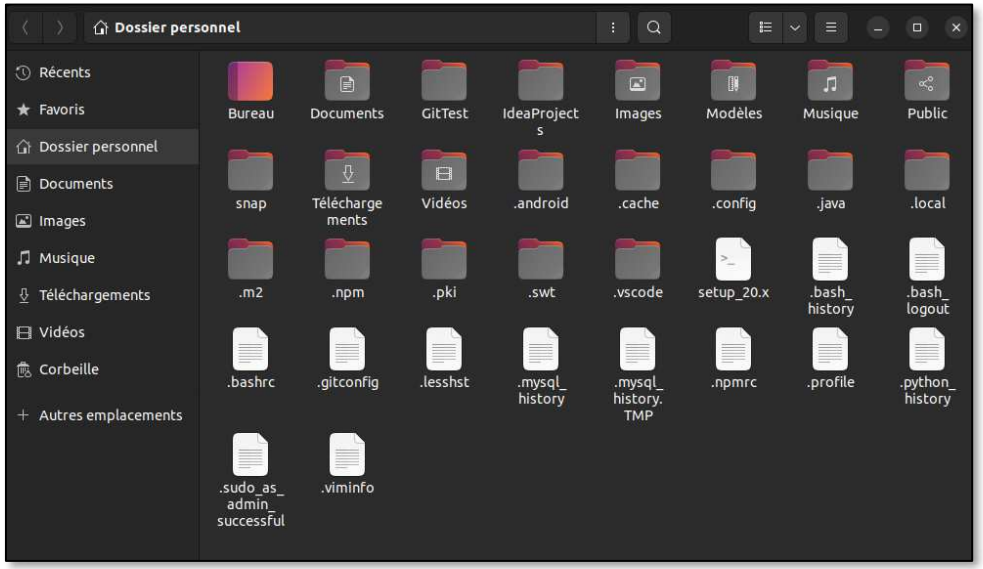
SystemConfigurator.set_gnome(proxy)      # Configuration du proxy de GNOME

GitConfigurator.enable_gcm()             # Configuration de GCM pour Git
GitConfigurator.set_user(name, email)    # Configuration informations utilisateur Git
#GitConfigurator.set_proxy(proxy)         # Configuration proxy de Git

DevToolsConfigurator.set_maven(proxy)    # Configuration du proxy de Maven
DevToolsConfigurator.set_gradle(proxy)   # Configuration du proxy de Gradle
```

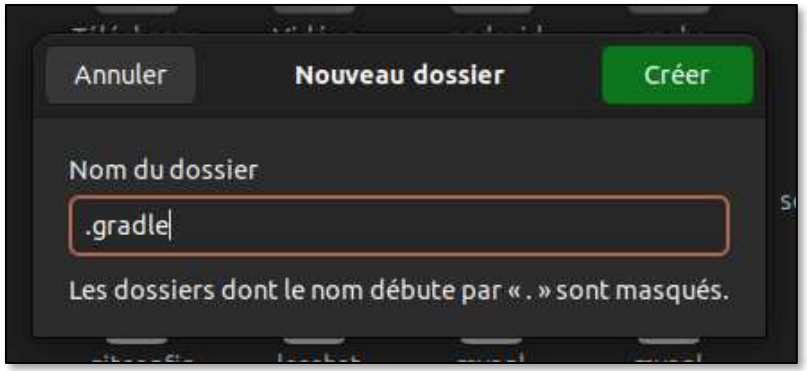
Spring Boot & IntelliJ

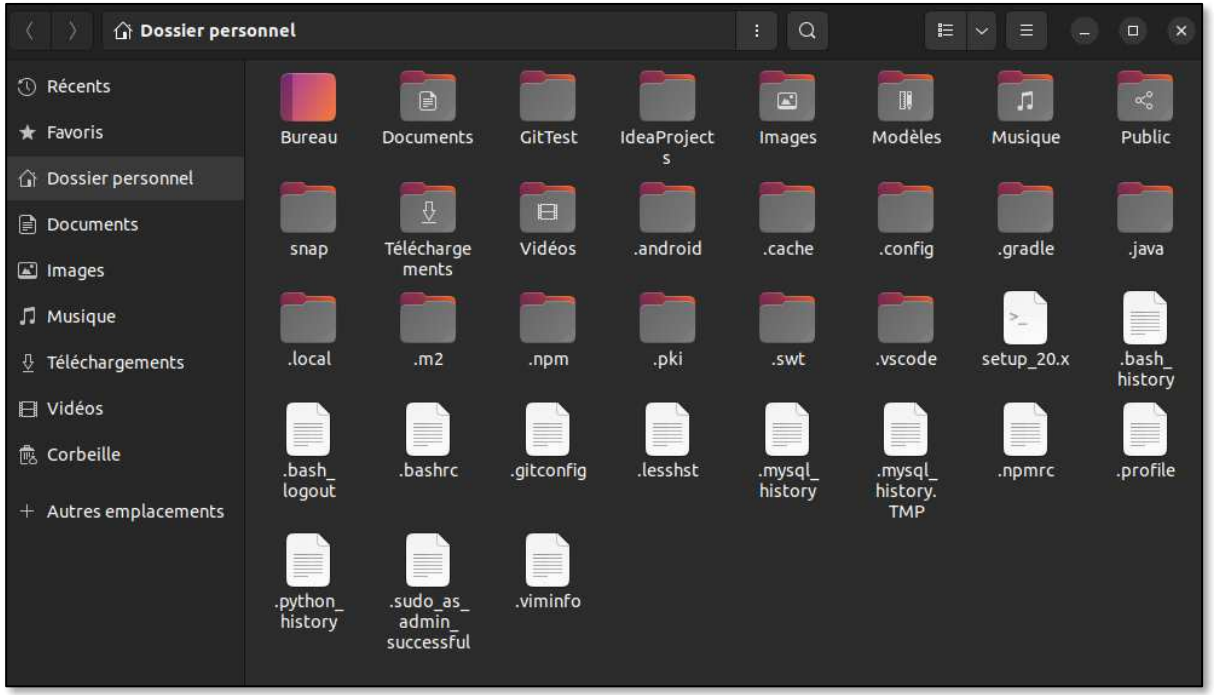
- **Cas du travail avec un proxy :**
  - Pour réaliser ou vérifier la configuration du proxy vous allez devoir ouvrir votre dossier **home** :
  - Il faut que les dossiers cachés soit affichés (les dossiers cachés commencent par un point).
  - Si les dossiers cachés ne sont pas visibles, taper simultanément sur **[CTRL] + [H]**



Spring Boot & IntelliJ

- **Cas du travail avec un proxy :**
  - S'il n'existe pas, créer le dossier caché : **`.gradle`**





Spring Boot & IntelliJ

- **Cas du travail avec un proxy :**
  - Entrer dans le dossier **`.gradle`**
  - Créer un nouveau fichier (s'il n'existe pas) : **`gradle.properties`**
  - Renseigner votre proxy et son port avec les 4 lignes suivantes :

```
systemProp.http.proxyHost=172.16.0.51
systemProp.http.proxyPort=8080
systemProp.https.proxyHost=172.16.0.51
systemProp.https.proxyPort=8080
```

  - **Astuces :**
    - Vous pouvez retrouver votre proxy avec la page : http://wpad.vinci-melun.org/wpad.dat
    - Pour créer votre fichier vous pouvez soit taper dans un terminal la commande :
      - **`touch gradle.properties`**
    - Vous pouvez également générer le fichier à partir de **VSCode** ou **IntelliJ**

Redémarrer IntelliJ pour qu'il prenne en compte les changements.

Initialiser un projet Spring Boot

avec **Spring Initializr**

Spring Initializr

- **Pourquoi utiliser Spring Initializr ?**

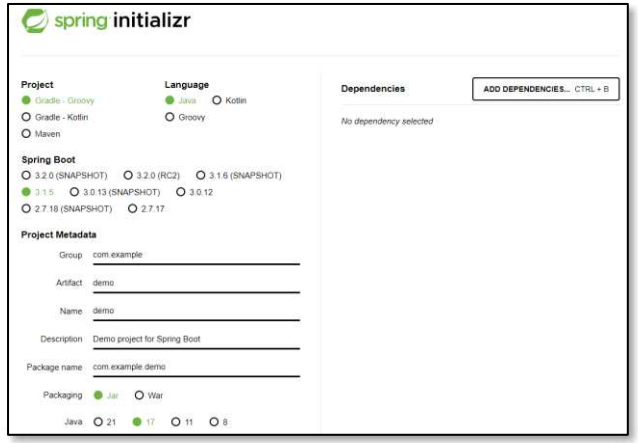
  - Initialiser un Frameworks comme **Spring Boot** est une tâche complexe qui nécessite des connaissances.

  - Heureusement nous avons **Spring Initalizr** pour nous aider dans cette tâche.

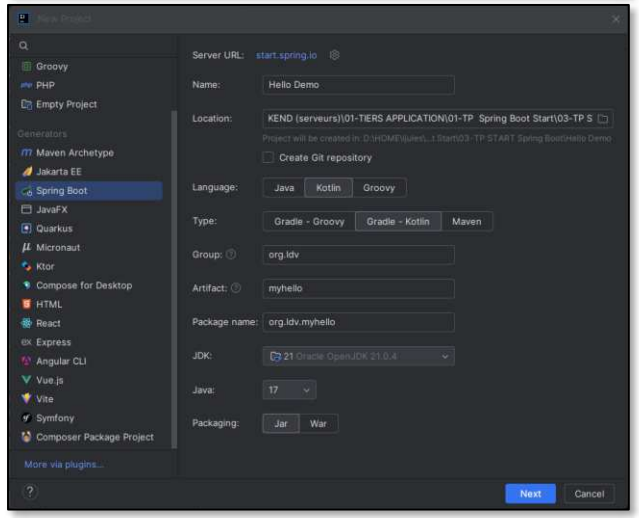
Spring Initializr

- **Pourquoi utiliser Spring Initializr ?**

  - **Spring Initializr** est accessible soit en ligne :
    - https://start.spring.io

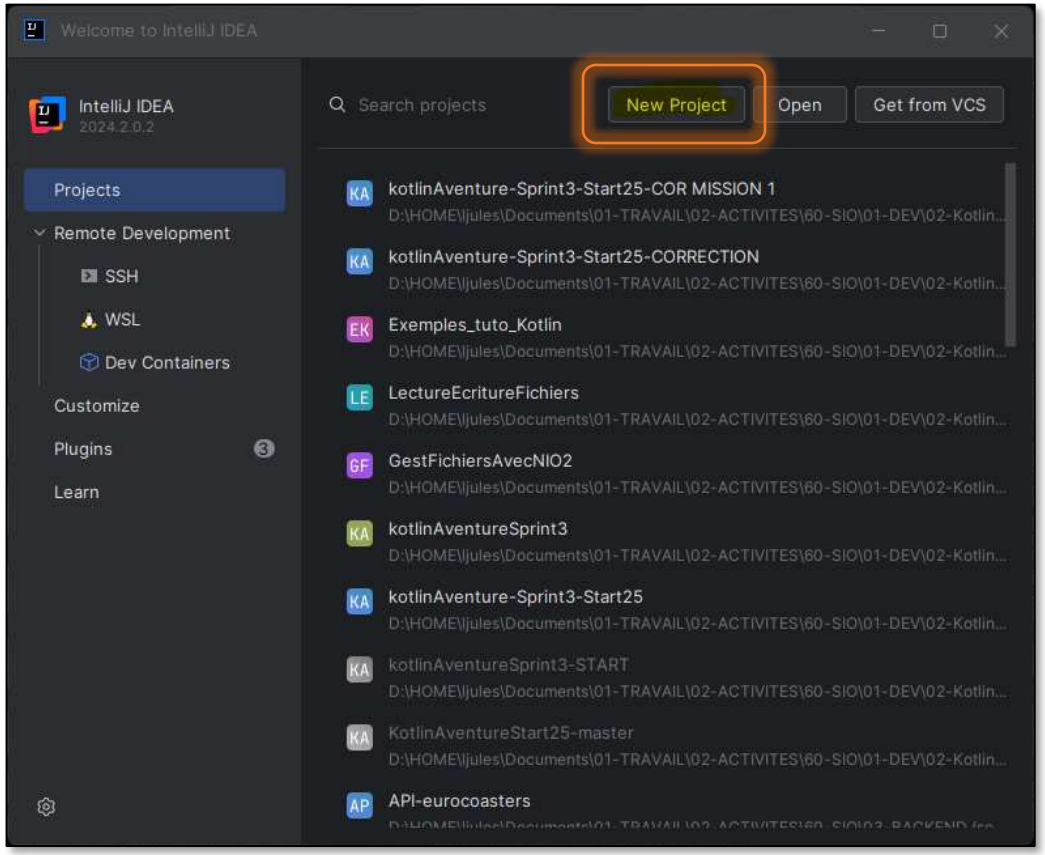


  - Mais également accessible depuis **IntelliJ** :



Spring Initializr

- **Application**
  - Ouvrir **IntelliJ** et lancer un nouveau projet



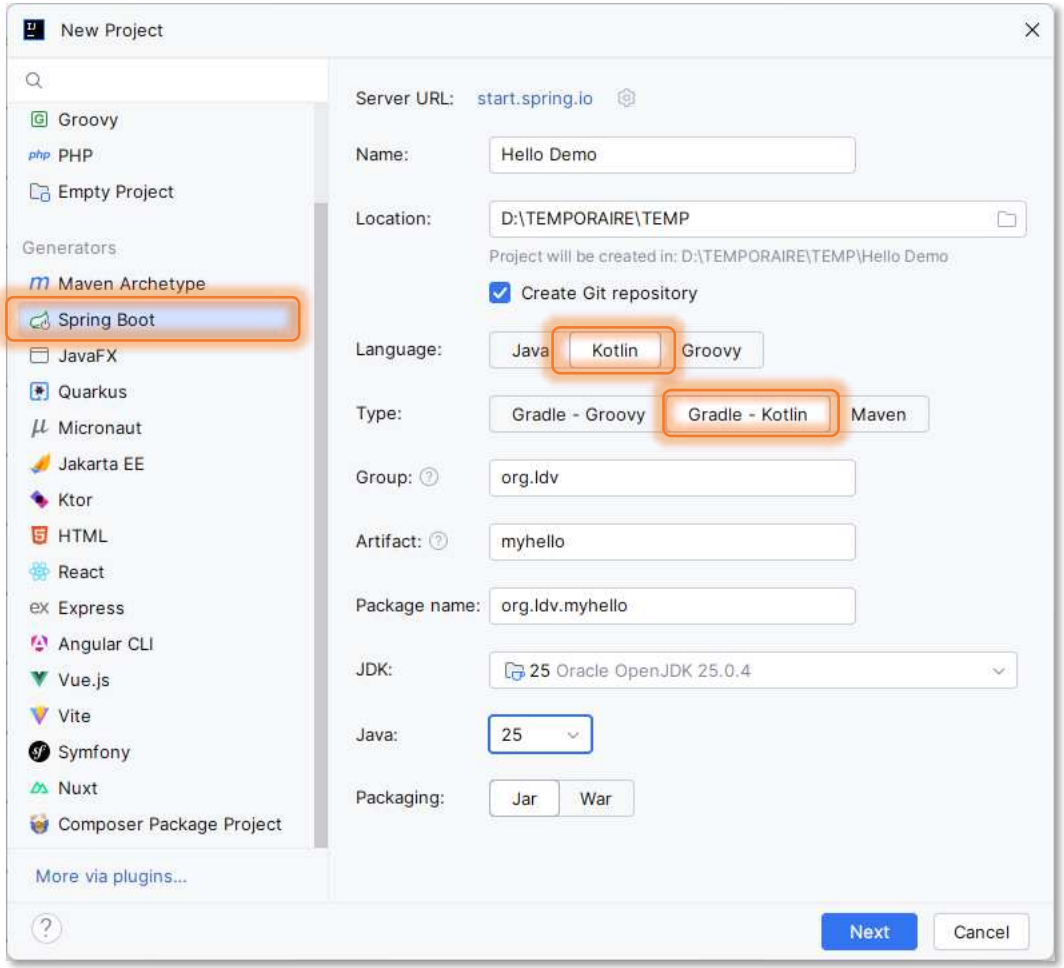
Page suivante pour la suite de la configuration

Spring Initializr

- Cliquer sur **Spring Boot** dans la rubrique **Generator**

- **Compléter les informations**
  - **Name :** *Hello Demo*
  - **Language :** *Kotlin*
  - **Type :** *Gradle - Kotlin*
  - **Group :** *org.ldv*
  - **Artifact :** *myhello*
  - **Package name :** *org.ldv.myhello*
  - **JDK :** *25*
  - **Java :** 25
  - **Packaging :** Jar

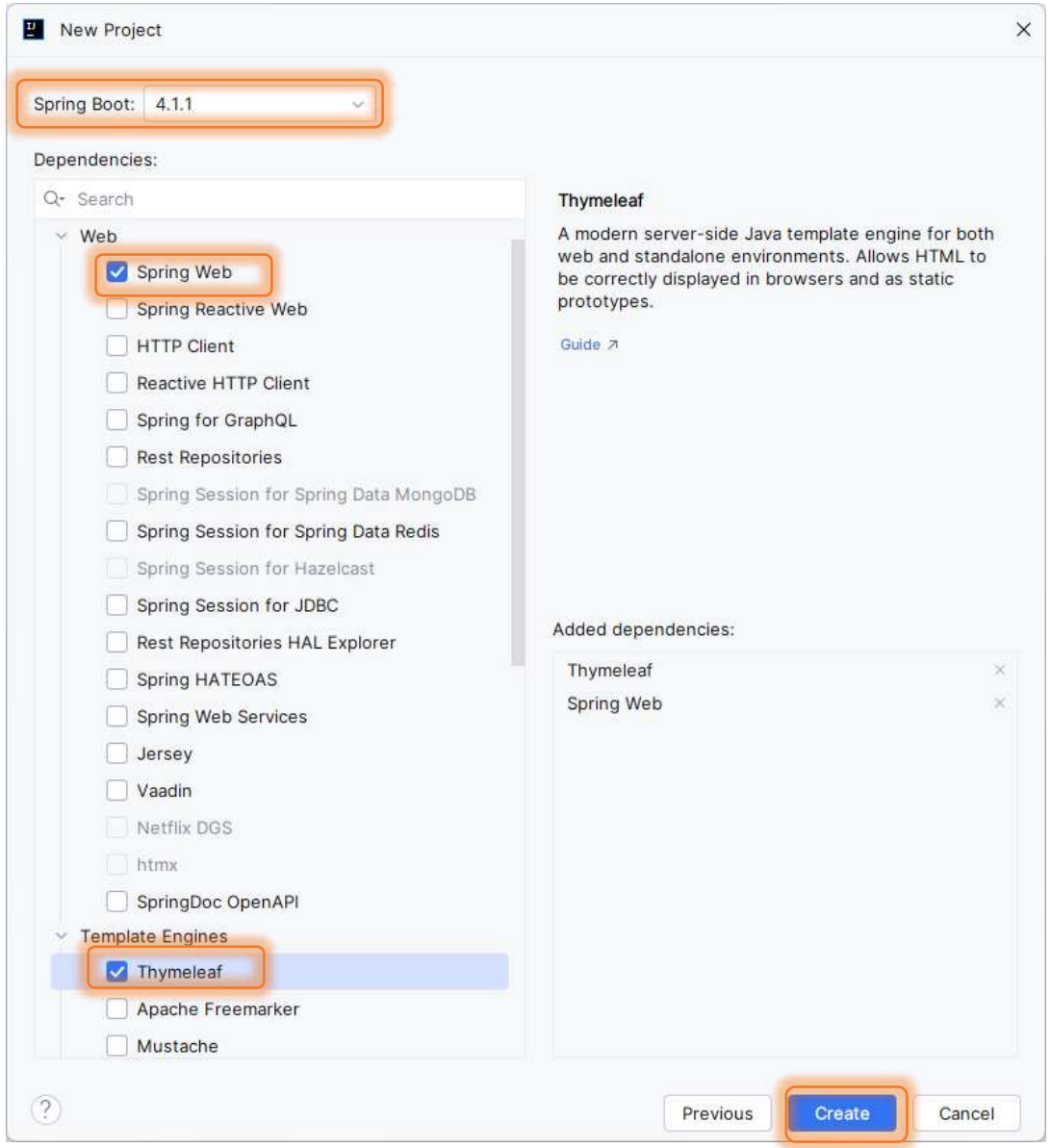
Page suivante pour la suite de la configuration



Spring Initializr

- **Suite de la configuration :**
  - **Spring Boot version :** *4.1.1*
  - **Ajouter les dépendances suivantes :**
    - **Spring Web** [WEB]
    - **Thymeleaf** [TEMPLATE ENGINES]

Cliquer sur [Create]
pour générer votre projet

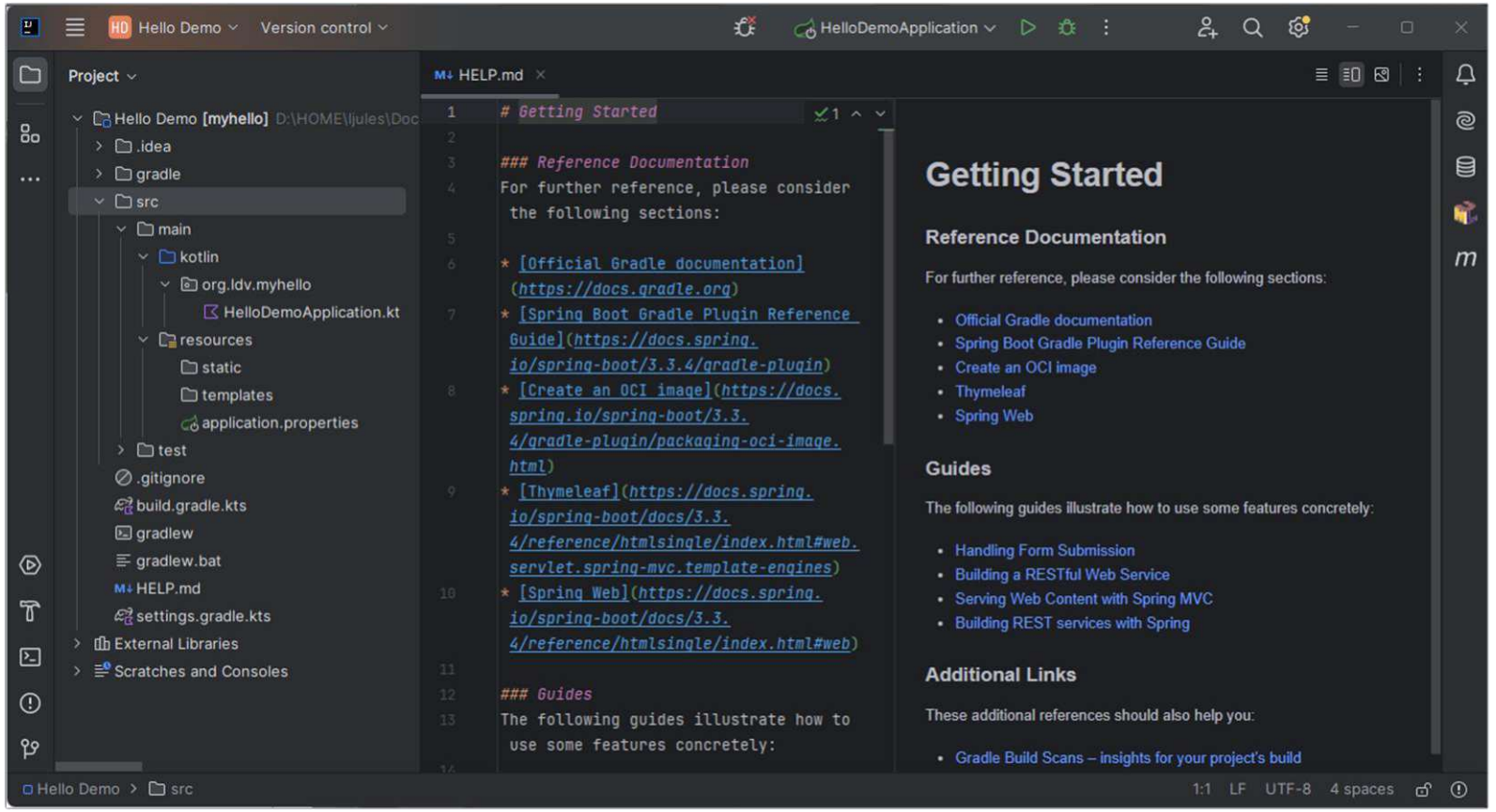


Première exécution

Spring Boot & IntelliJ

Spring Boot & IntelliJ

- **Démarche à suivre :**
  - Vous devez à ce stade avoir cette configuration



Spring Boot & IntelliJ

- **Analyse de la structure du projet**
  - Il est important de se repérer dans l'arborescence du projet :



Dossier accueillant les assets : fichiers css, images, etc.

Fichier de configuration de l'application : proxy, identification, information connexion BD, etc.

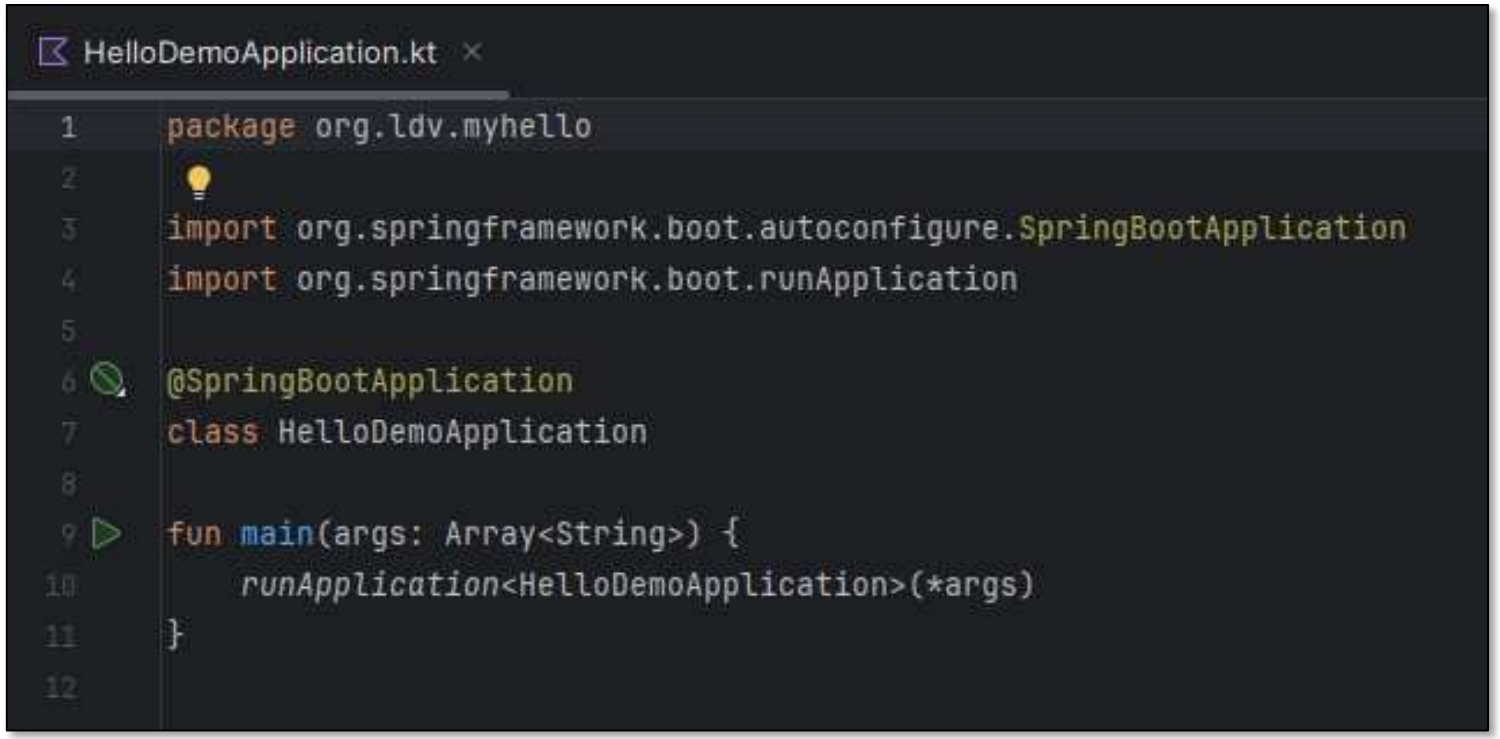
Application principale (fonction **`main()`**)

Dossier stockant les fichiers du moteur de templates : **Thymeleaf /Freemarker**

Fichier de configuration **Gradle**

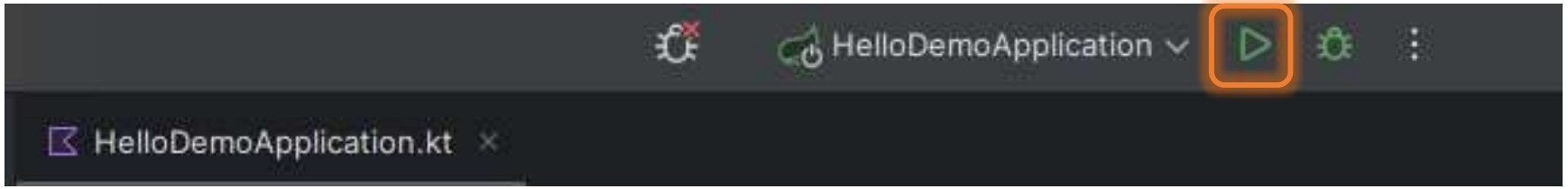
Spring Boot & IntelliJ

- **Ouvrons le fichier HelloDemoApplication.kt :**
  - Ce fichier contient le point d'entrée du programme (fonction **`main()`**).

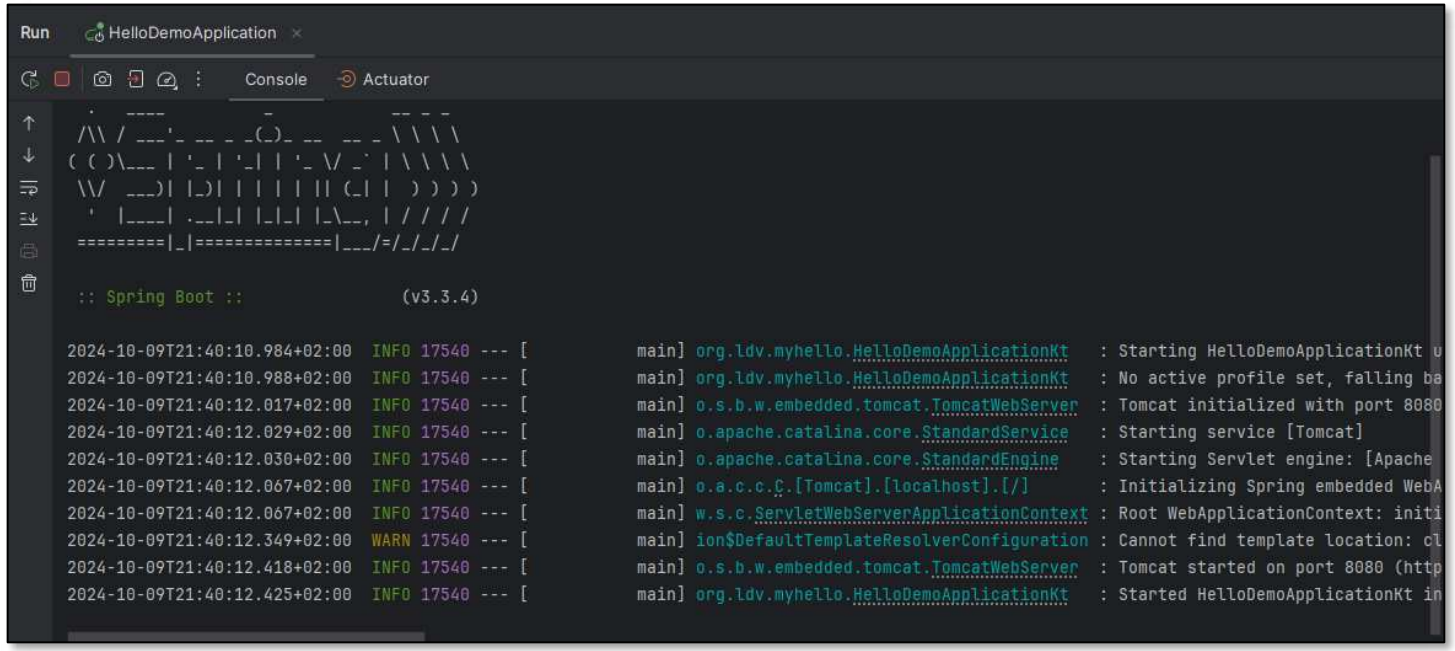


Spring Boot & IntelliJ

- **Exécuter votre application Spring Boot :**



- Vous devez avoir le log se dérouler dans le terminal d'**IntelliJ** :

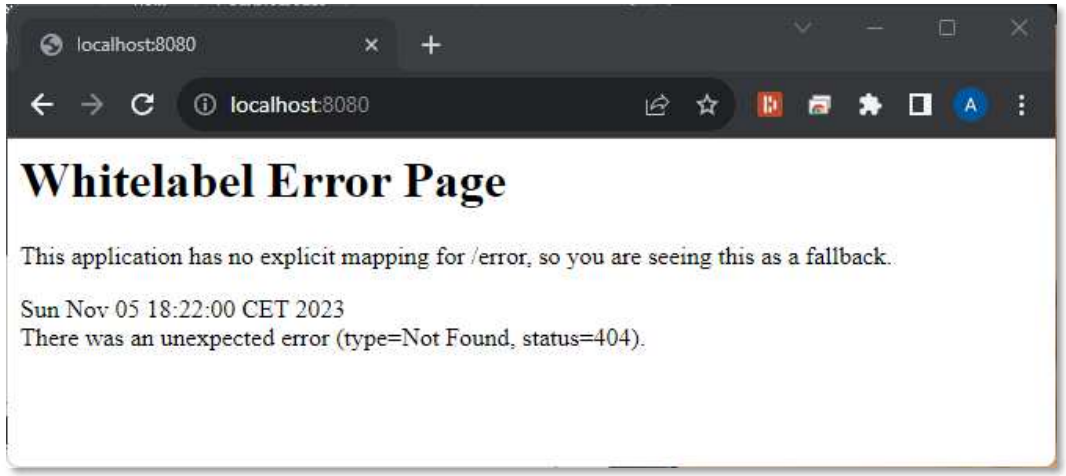


Spring Boot & IntelliJ

- **Vérification de la mise en service du serveur TomCat :**
  - Même si vous n'avez pas encore créé de contenu, l'exécution de l'application lance automatiquement un serveur web **TomCat**.
  - Par défaut ce serveur est exécuté sur port : **8080**

  - Nous pouvons vérifier cela en ouvrant un navigateur avec l'URL suivante :
    - **http://localhost:8080/**

    - **Vous devez obtenir la page suivante :**



Spring Boot & IntelliJ

- **Configuration du port du serveur :**
  - En cas de conflit sur le port, **Tomcat** le signal clairement dans l'interface d'**IntelliJ**.

```
***************************
APPLICATION FAILED TO START
***************************

Description:

Web server failed to start. Port 8090 was already in use.

Action:

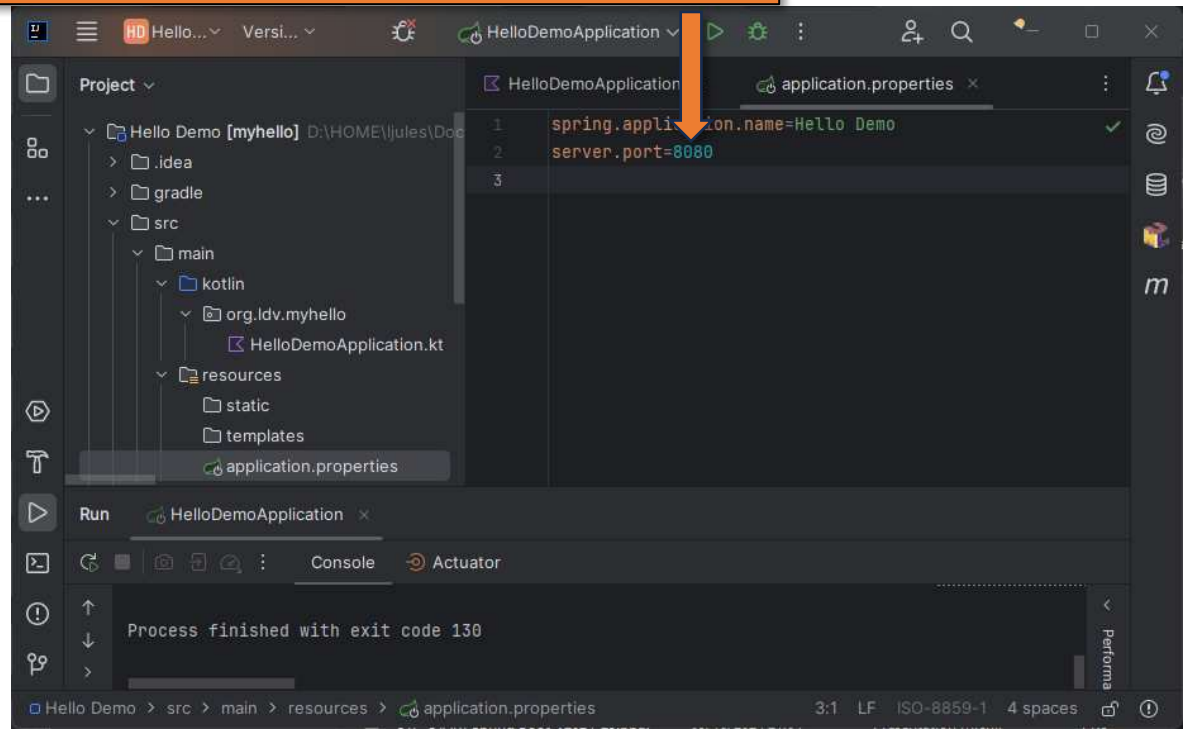
Identify and stop the process that's listening on port 8090 or configure this application to listen on another port.


Process finished with exit code 0
```

Spring Boot & IntelliJ

- **Configuration du port du serveur :**
  - Il est possible de spécifier le numéro de port.
  - Il suffit de le renseigner dans le fichier : **application.properties**

server.port=[port-number]



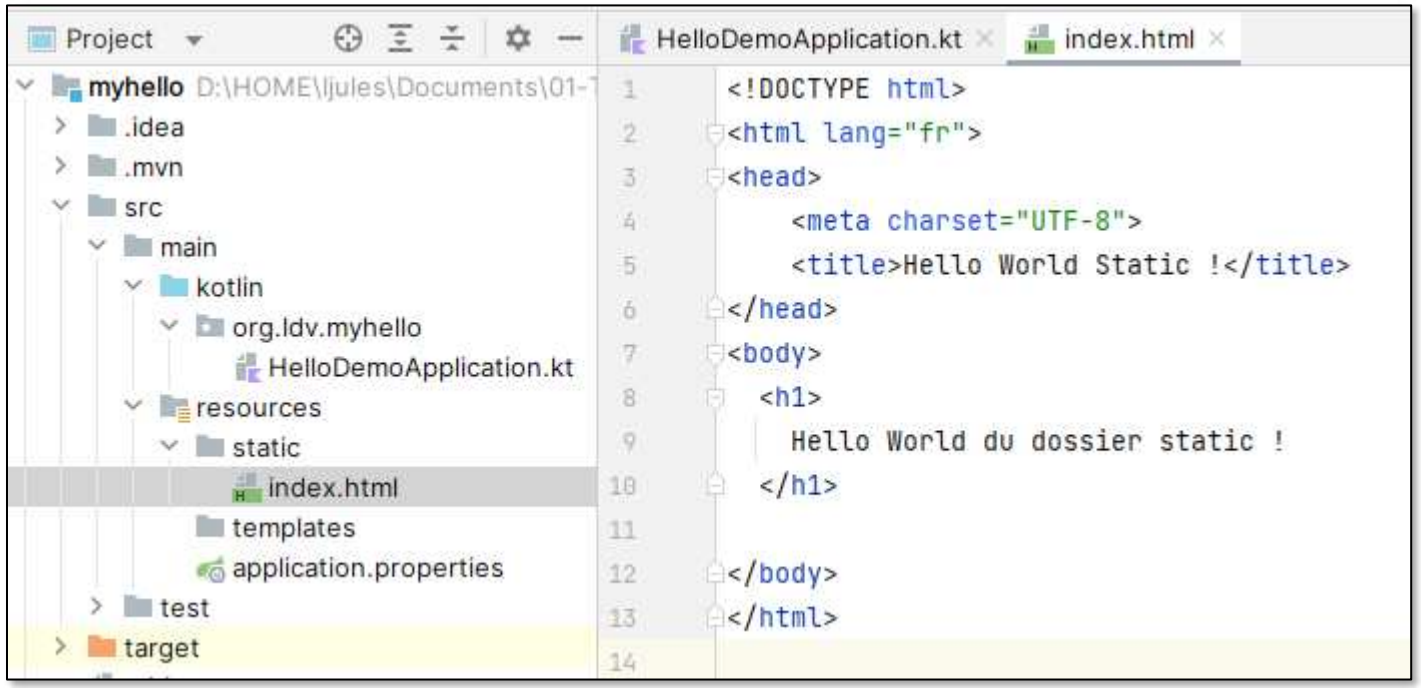
Pour info.

Personnalisation de notre application

Static « Hello World ! »

Static Hello World !

- **Test d'une page HTML statique :**
  - La page d'accueil générée par **TomCat** n'est pas très avenante.
  - Nous allons voir le comportement de notre application si on génère un simple fichier **`index.html`** que nous allons placer dans le dossier **`static`**.
  - Stopper votre application, et créer le fichier **`index.html`** avec le contenu suivant :



Static Hello World !

- **Test d'une page HTML statique :**
  - Exécuter de nouveau votre application et réouvrir la page : http://localhost:8080/
  - Vous devez maintenant avoir la page suivante qui s'affiche :



Static Hello World !

- **Test d’une page HTML statique :**
  - A ce stade nous savons que notre machine est opérationnelle.
  - Nous allons dans les pages suivantes apporter un peu de dynamisme à notre site Web !

Personnalisation de notre application

Hello World du contrôleur !

Hello Contrôleur

- **Objectif :**

  - Le « **Hello World ! » r**éalisé durant la partie précédente est rudimentaire puisque nous nous contentons d'utiliser **Spring Boot** comme un simple serveur web pour site statique.

  - Nous vous proposons de mettre de côté votre fichier **`index.html`** en le renommant en **`index_old.html`** par exemple.

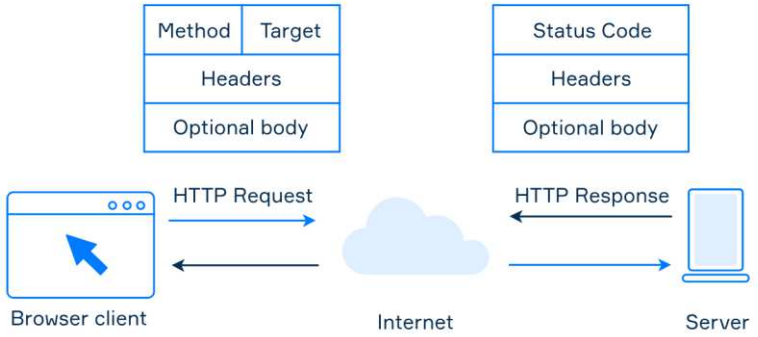
  - Nous allons donc dans cette partie tirer parti du **contrôleur** du modèle **MVC.**

Hello Contrôleur

- **Objectif :**

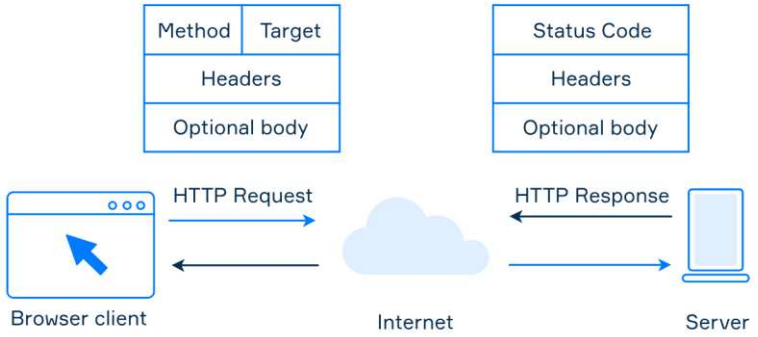
  - Le **Contrôleur** a pour objectif de traiter les requêtes rentrantes, d'appeler l'objet métier concerné de la couche **Modèle** et d'envoyer au client la réponse produite par la **Vue**.

  - Dans cette partie nous allons laisser le **Contrôleur** produire par lui seul la réponse.



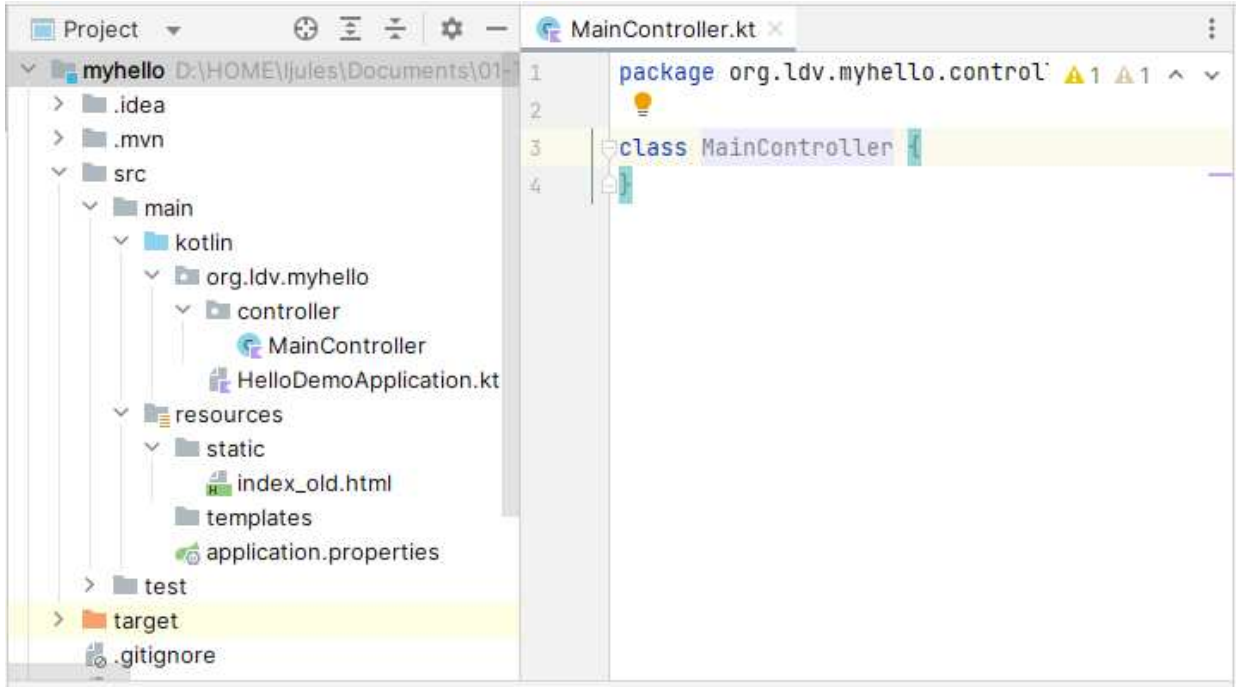
Hello Contrôleur

- **Objectif :**
  - L'idée est qu'il va renvoyer la donnée demandée par notre client.
  - Chaque fois que vous saisissez une **URL** et que vous la validez dans votre navigateur, ce dernier élabore une requête **GET** qui est interceptée par le serveur **TomCat**.
  - La requête est traitée par notre serveur.



Hello Contrôleur

- **Création du package et du fichier :**
  - Dans l'arborescence rajouter la package **`controller`**
  - Puis dans le package **`controller`** créer une nouvelle classe **Kotlin** appelée **`MainController`**



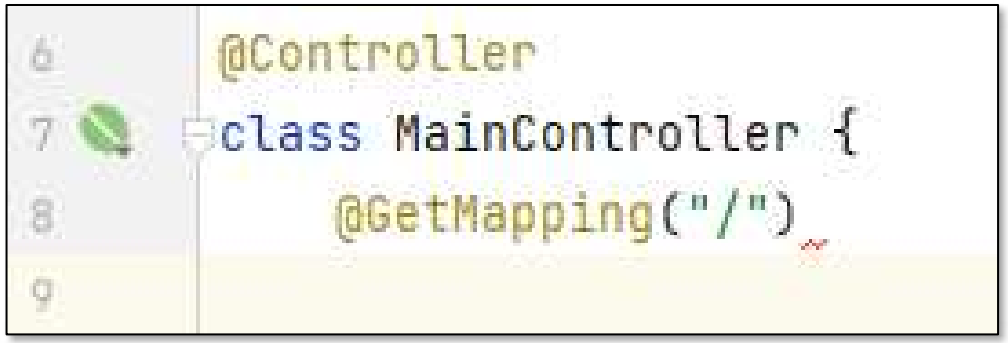
Hello Contrôleur

- **Ajout des annotations :**
  - Nous allons maintenant indiquer à Spring Boot que nous désirons créer un **contrôleur** avec l'annotation **`@Controller`** :



Hello Contrôleur

- **Ajout des annotations et du endpoint :**
  - Maintenant nous allons ajouter dans le corps de notre classe l'annotation **@GetMapping**.
  - Comme son nom l'indique, nous allons indiquons à **SpringBoot** que notre classe gère une requête **GET**. Mais ici l'annotation devra être complétée par un point terminal (**endpoint**), c'est tout simplement la fin de l'**URL** qui sera utilisée par le client pour accéder à la ressource gérée par notre 1er contrôleur.
  - Le **endpoint** sera tout simplement ici la racine de notre site, soit **« / »**



Hello Contrôleur

- **Elaboration de la réponse :**
  - Notre code est incomplet, c'est pour cela qu'il y a du rouge juste après la seconde annotation…
  - **SpringBoot** attend la réponse à fournir.
  - Pour ce faire on utilise l'annotation **@ResponseBody**, comme son nom l'indique, il indique à **Spring Boot** que la réponse à notre **endpoint** sera un corps de requête (*body*). Et que cette réponse sera directement produite par le **Contrôleur** et que nous ne utiliserons pas la **Vue**.

```
@Controller
class MainController {
    @GetMapping("/")
    @ResponseBody

}
```

Hello Contrôleur

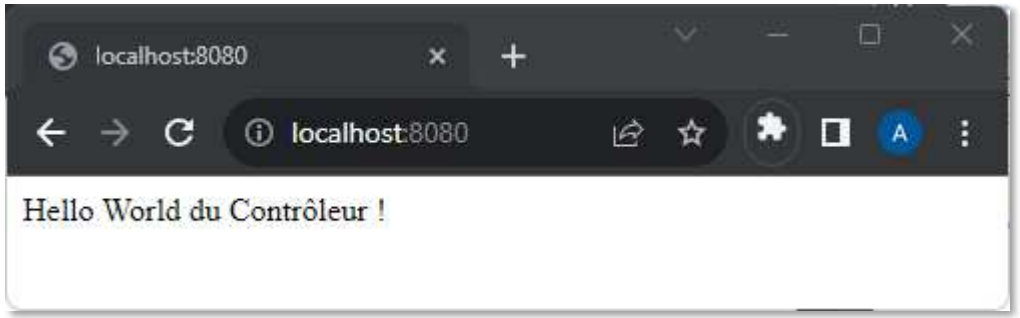
- **Elaboration de la réponse :**
  - Il faut maintenant élaborer notre réponse, qui sera ici tout simplement un **String** :

    « Hello World du Contrôleur ! »

    Pour produire cette réponse il suffit de rédiger une méthode à la suite de l'annotation.

    L'identifiant de la méthode n'a pas d'influence, l'inversion de contrôle opère sur notre code grâce à l'annotation **@ResponseBody**

```
7   @Controller
8   class MainController {
9       @GetMapping("/")
10      @ResponseBody
11      fun myResponse(): String {
12          return "Hello World du Contrôleur !"
13      }
14  }
```



Redémarrer l'application pour que les modifications prennent effet.

Hello Contrôleur

- **Elaboration de la réponse :**
  - Cette évolution du code ne fait pas encore intervenir la **Vue**, mais c'est avec ce genre de **Contrôleur** que nous pourrons implémenter des **API REST**. Une **API** étant une « interface » qu'expose notre serveur pour fournir des réponses au format **JSON**.
  - La partie élaboration de la réponse est alors fournie par la couche **Modèle** grâce à un composant que nous désignerons service… Nous verrons un exemple de tout cela dans le prochain TP.

```
7   @Controller
8   class MainController {
9       @GetMapping("/")
10      @ResponseBody
11      fun myResponse(): String {
12          return "Hello World du Contrôleur !"
13      }
14  }
```

Personnalisation de notre application

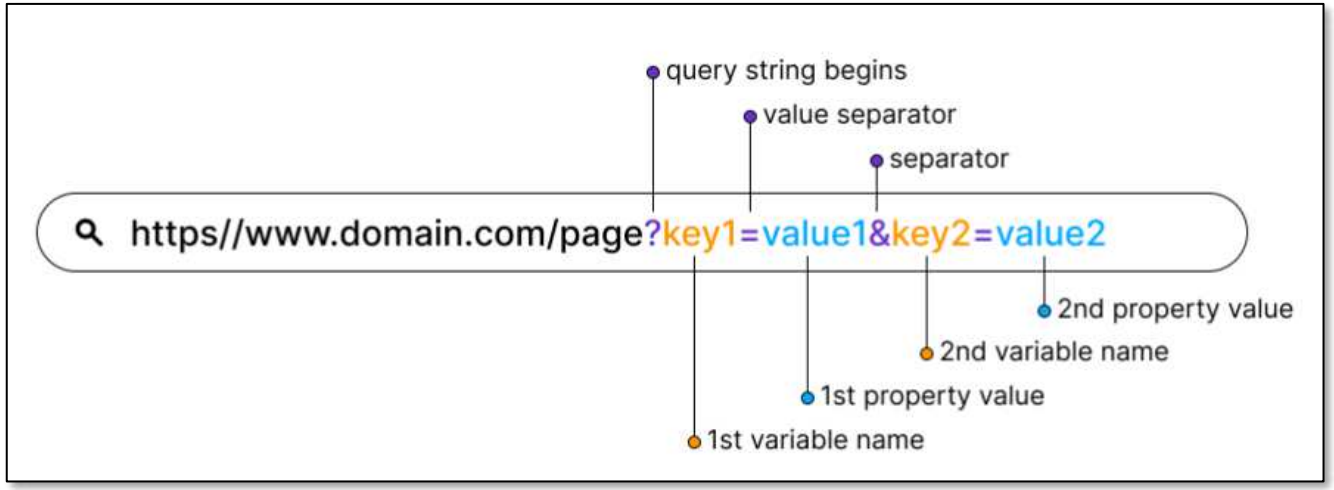
Transmission de données par le client

Transmission de données par le client

- **Rappel sur les dispositions du HTTP pour transmettre des données**
  - HTTP nous offre 2 possibilités pour transmettre des données :
    - **Les chaînes de requêtes (*QuerryString*) :**
      - On passe des données avec une requête **GET**, les données sont directement incluses dans l'**URL**.
      - Nous allons exploiter cette possibilité pour illustrer la réception des données du client par notre application.
    - **Les données dans le corps d'une requête POST :**
      - Le cas classique est l'envoi d'un formulaire par l'utilisateur qui met en œuvre une requête **POST**.
        - Pour le moment nous ne pouvons pas utiliser cette méthode car nous n'avons pas de **Vue** pour fournir un formulaire à remplir.
      - Mais les requêtes **POST** nous permettent d'envoyer d'autres formes de données :
        - Transfert de fichiers;
        - Données au format **JSON**.
        - Etc.

Transmission de données par le client

- **Exemple d'une donnée transmise par QuerryString :**
  - Un petit rappel sur le format d'une **URL** suivie d'une chaîne de requête :
  - Le symbole **?** Indique le début d'une **QuerryString**.
  - La **QuerryString** est constitué de paires de clés-valeurs (à un peu comme un Map).
  - Le format d'une clé-valeur utilise le symbole **=** : **clé=valeur**
  - Les différentes paires clés-valeurs sont séparé par le symbole **&**



Transmission de données par le client

- **Exemple d'une donnée transmise par QuerryString :**
  - Pour nous initier au traitement d'une **QuerryString** nous vous proposons de renseigner votre nom grâce à une **URI** au format suivant :
    - http://localhost:8080/hello?nom=John
  - On attend que notre serveur produise la réponse :
    - **`Hello John !`**

Transmission de données par le client

- **Exemple d'une donnée transmise par QuerryString :**

  - Nous allons réaliser une nouvelle méthode qui sera associée/mappée à un nouveau point terminal/endpoint : **« /hello »**

  - La méthode prendra un argument qui va nous permettre de récupérer les **clés/valeurs** de la **QuerryString**.

  - C'est **SpringBoot** qui fournira l'argument à notre méthode ! Le paramètre à créer dans l'entête de notre méthode devra est du type **`HttpServletRequest`**.

  - Enfin nous disposerons de la méthode **`getParameter`** (méthode de l'objet de type **`HttpServletRequest`**).

Transmission de données par le client

- **Exemple d'une donnée transmise par QuerryString :**

  - En pratique notre **Contrôleur** devient :

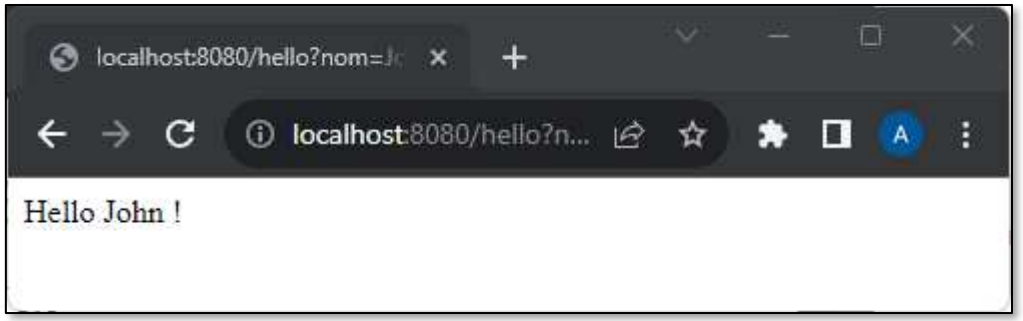
```
@Controller
class MainController {
    @GetMapping("/")
    @ResponseBody
    fun myResponse(): String {
        return "Hello World du Contrôleur !"
    }
    @GetMapping("/hello")
    @ResponseBody
    fun querryString(querry: HttpServletRequest): String{
        val nom = querry.getParameter("nom") ?: ""
        return "Hello ${nom} !"
    }
}
```

Transmission de données par le client

- **Exemple d'une donnée transmise par QuerryString :**

  - Exécuter cette nouvelle version de votre Contrôleur est lancer l'**URL** pour procéder au test : **http://localhost:8080/hello?nom=John**

```
@Controller
class MainController {
    @GetMapping("/")
    @ResponseBody
    fun myResponse(): String {
        return "Hello World du Contrôleur !"
    }
    @GetMapping("/hello")
    @ResponseBody
    fun querryString(querry: HttpServletRequest): String{
        val nom = querry.getParameter("nom") ?: ""
        return "Hello ${nom} !"
    }
}
```



Transmission de données par le client

- **Ajout d'une seconde clé/valeur :**

  - Nous allons améliorer notre méthode pour que cette dernière puisse traiter le **nom** et le **prénom**.
  - L'**URL** à traiter devient :
  - http://localhost:8080/hello?nom=DOE&prenom=John

  - Et le résultat produit attendu est le suivant :



Transmission de données par le client

- **Ajout d'une seconde clé/valeur :**
  - Apportez les modifications nécessaires et tester votre solution.

```
@GetMapping("/hello")
@ResponseBody
fun querryString(querry: HttpServletRequest): String{
    val nom = querry.getParameter("nom") ?: ""
    TODO
    return "Hello TODO ${nom} !"
}
```

Transmission de données par le client

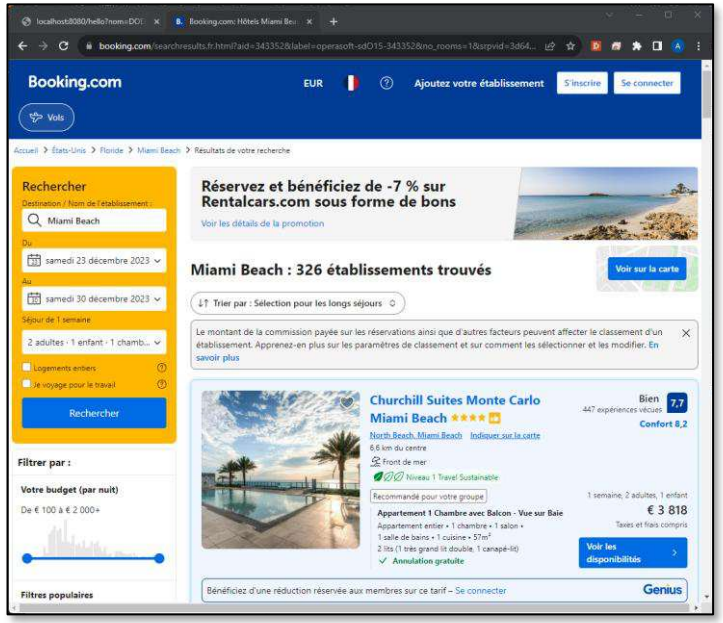
- **Récupérer une collection de valeurs :**
  - L'usage de **QuerryString** est une pratique courante.
  - Il faut juste que les informations transmises ne soient pas sensibles en termes de sécurité, informations personnelles, etc.
  - L'un des avantages à utiliser des **QuerryString**, c'est que du point de vue utilisateur, ce dernier pourra enregistrer des **URL** (bookmark) durant sa navigation.

Transmission de données par le client

- **Récupérer une collection de valeurs :**

  - Un cas d'usage typique : *« Je désire partir cette fin d'année au soleil à Miami, j'utilise pour cela le site **Booking** pour chercher un lieu sympathique. »*
  - Une fois les informations envoyées : *dates*, *lieux*, *personnes,* etc, l'utilisateur peut conserver sa requête pour l'exécuter plus tard.

  - Exemple de requête pour le site **Booking** : requêteBooking



Transmission de données par le client

- **Récupérer une collection de valeurs :**

  - Le problème est la longueur de la chaîne qui est conséquente !

  - A ce moment nous avons tout intérêt à récupérer les clés/valeurs de la **QuerryString** dans une collection.

  - Une première variante pour notre **Contrôleur** est l'utilisation d'un tableau associatif (Map).

  - Pour ce faire nous allons utiliser l'injection de dépendance directement en argument de notre méthode avec l'annotation `@RequestParam`

Transmission de données par le client

- **Récupérer une collection de valeurs :**
  - Voici le code à tester :

```
@GetMapping("/hello")
@ResponseBody
fun querryString(@RequestParam querry: Map<String, String>): String{
    val nom = querry["nom"] ?: ""
    val prenom = querry["prenom"] ?: ""
    return "Hello ${prenom} ${nom} !"
}
```

Transmission de données par le client

- **Traiter les URI de façon dynamique :**

  - **SpringBoot** nous permet d'insérer des éléments variables dans nos **endpoints**.

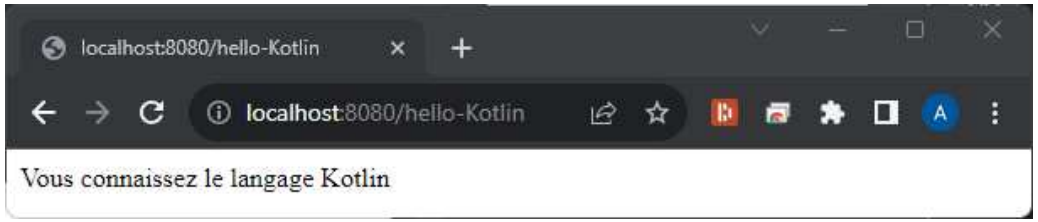
  - Il suffira de rédiger les **endpoints** en plaçant les éléments variables de votre **endpoints** entre une paire d'accolades **{ … }**.

  - L'annotation **`@PathVariable`** placé en paramètre de notre méthode va nous permettre d'associer une variable à l'élément dynamique de notre **endpoint**.

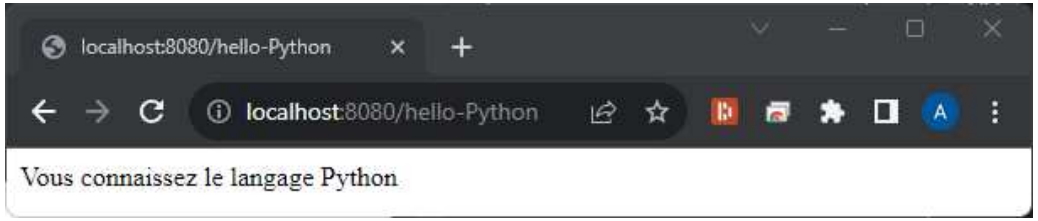
Transmission de données par le client

- **Traiter les URI de façon dynamique :**
  - **Exemples :**
    - Nous avons une **URL** qui correspond à différent langage de programmation.
    - Notre programme pour la démo va extraire la fin de l'**URL** pour afficher le langage comme ci-dessous :

- http://localhost:8080/hello-Kotlin



- http://localhost:8080/hello-Python



- http://localhost:8080/hello-Java



Transmission de données par le client

- **Traiter les URI de façon dynamique :**
  - **Exemples :**
    - Voici le code du **Contrôleur** correspondant :

```
@Controller
class MainController {
    @GetMapping("/hello-{language}")
    @ResponseBody
    fun findLanguage(@PathVariable("language") language: String,
                     request: HttpServletRequest): String {
        return "Vous connaissez le langage ${language}"
    }
}
```

Collaboration Vue – Contrôleur

La couche Vue

La couche Vue

- **Pourquoi la Vue ?**

  - Pour le moment nous avons uniquement utilisé la couche **Contrôleur**.

  - Nous avons demandé au Contrôleur de répondre à différentes requêtes de type **GET** en envoyant un String dans le corps de la requête de réponse.

  - Nous pourrions tout à fait générer une page **HTML** complète dans la **`String`** retournée par notre méthode de **Contrôleur.** Mais cela n'est pas une bonne pratique. En effet car cela est justement le rôle/responsabilité de la couche **Vue**.

La couche Vue

- **Comment ?**

  - Dans cette partie consacrée à la **Vue**, nous allons voir :

    - Une première initiation à un moteur de template (template engine)

    - Comment fait collaborer nos 2 couches : **Vue** & **Contrôleur**

La couche Vue

- **Le moteur de template :**

  - Le rôle de la **Vue** est de produire à minima des pages **HTML** de façon dynamique.

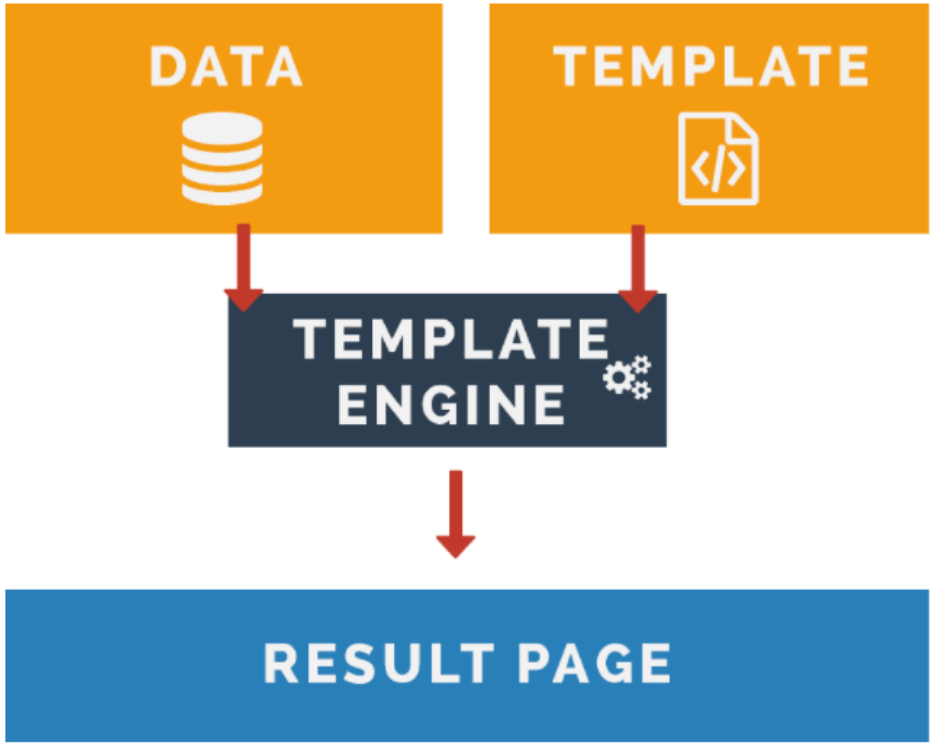
  - Nous entendons par dynamique que nos pages **HTML** seront composés d'éléments **HTML** statiques et d'élément **HTML** variables/dynamique.

  - Pour ce faire nous utilisons un **moteur de template**.

La couche Vue

- **Le moteur de template :**

  - **Le moteur de template repose :**
    - Sur un langage de description permettant d'élaborer les pages **HTML** à partir de données (variables divers). Cela correspond plus ou moins à un template.
    - D'un moteur (engine) qui va à partir de votre template et des données envoyées par le Contrôleur produire la page qui sera envoyée au client.

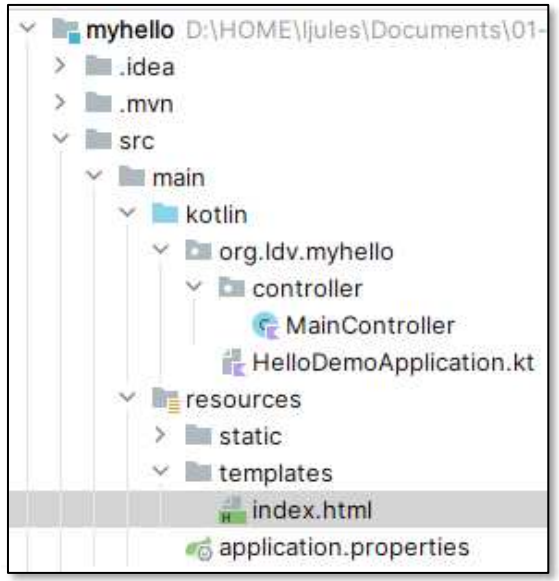


La couche Vue

- **Le moteur de template :**
  - **Choix du moteur de template :**
    - Durant la phase d'initialisation de notre projet **Spring Boot** nous avions ajouté un moteur de template, à savoir **Thymeleaf** .
    - Il existe d'autre choix : **Mustache**, **FreeMarker**
    - Le choix d'un moteur template est une question personnelle.

La couche Vue

- **Génération de notre 1ère vue :**
  - Nous allons créer notre 1er template, pour ce faire créer un simple fichier **HTML** que vous nommerez **`index.html`** et que vous devez impérative placer dans le dossier **`templates`** (vous pourrez à votre convenance créer des sous-dossiers à partir du dossier **`templates`**).
  - Rédiger le code de template comme ci-dessous :



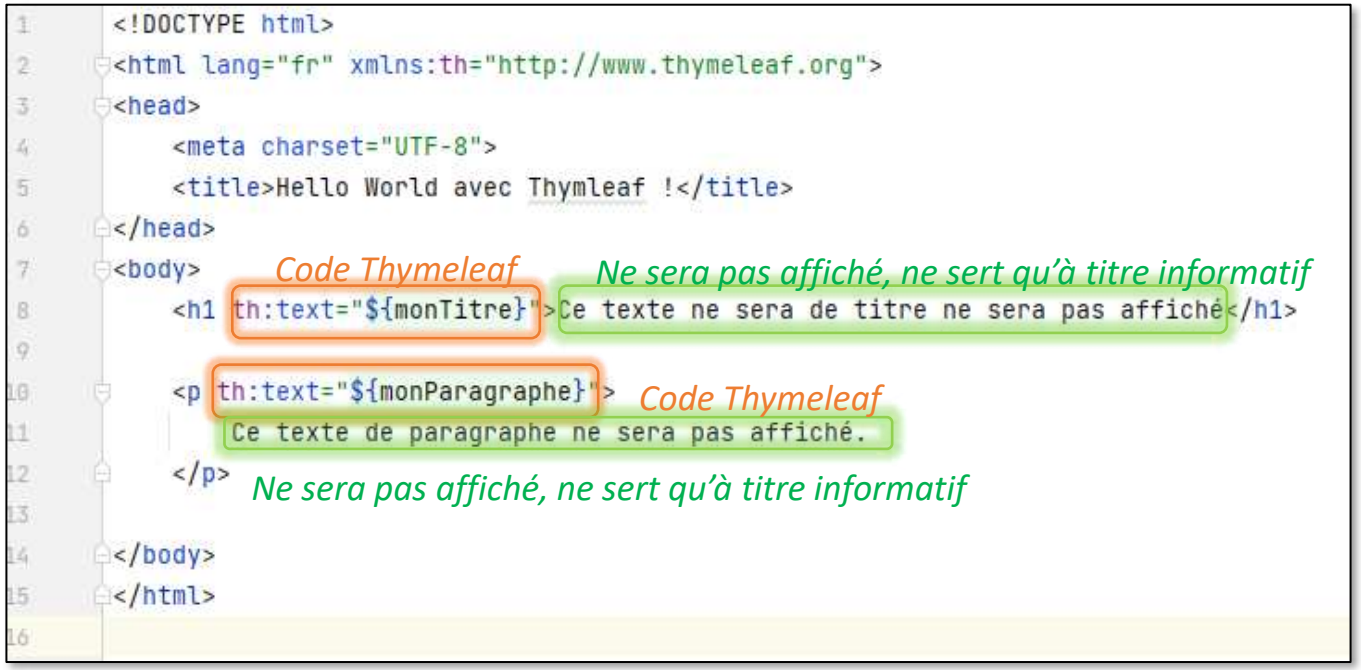

```
<!DOCTYPE html>
<html lang="fr" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Hello World avec Thymleaf !</title>
</head>
<body>
    <h1 th:text="${monTitre}">Ce texte ne sera de titre ne sera pas affiché</h1>

    <p th:text="${monParagraphe}">
        Ce texte de paragraphe ne sera pas affiché.
    </p>

</body>
</html>
```

La couche Vue

- **Génération de notre 1ère vue :**
  - Explication du contenu :
    - La grande majorité du langage de template **Thymeleaf** n'est que du simple langage **HTML**.
    - La nouveauté ici, sont l'insertion dans les balises **<h1>** et **<p>** des attributs spécifiques à **Thymeleaf** :
      - **th:text="${monTitre}"**
      - **th:text="${monParagraphe}«**
  - **Thymeleaf** nous permet d'insérer 2 variables du **Contrôleur :**
    - **monTitre**
    - **monParagraphe**




```
<!DOCTYPE html>
<html lang="fr" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Hello World avec Thymleaf !</title>
</head>
<body>
    <h1 th:text="${monTitre}">Ce texte ne sera de titre ne sera pas affiché</h1>

    <p th:text="${monParagraphe}">
        Ce texte de paragraphe ne sera pas affiché.
    </p>

</body>
</html>
```

La couche Vue

- **Génération de notre 1ère vue :**
  - La partie Contrôleur :
    - Nous allons créer un **Contrôleur** pour le endpoint **« / »** (racine de note site).
    - Avant d'élaborer le code du **Contrôleur**, rajouter les 2 imports suivants :

```
// Import pour la gestion de la Vue :
import org.springframework.ui.Model
import org.springframework.ui.set
```

La couche Vue

- **Génération de notre 1ère vue :**
  - La partie Contrôleur :

    - Nous allons appeler notre méthode de **Contrôleur** tout simplement **`index`** puisque que le mapping se fera à la racine de notre site et que historiquement la page d'accueil du site statique s'appelait par convention **index.html**, nom que nous avons également utilisé pour notre template.

    - Mais tout cela n'est qu'une convention pour nous les humains, pour **Spring Boot** ce sont les annotations qui font le job !

La couche Vue

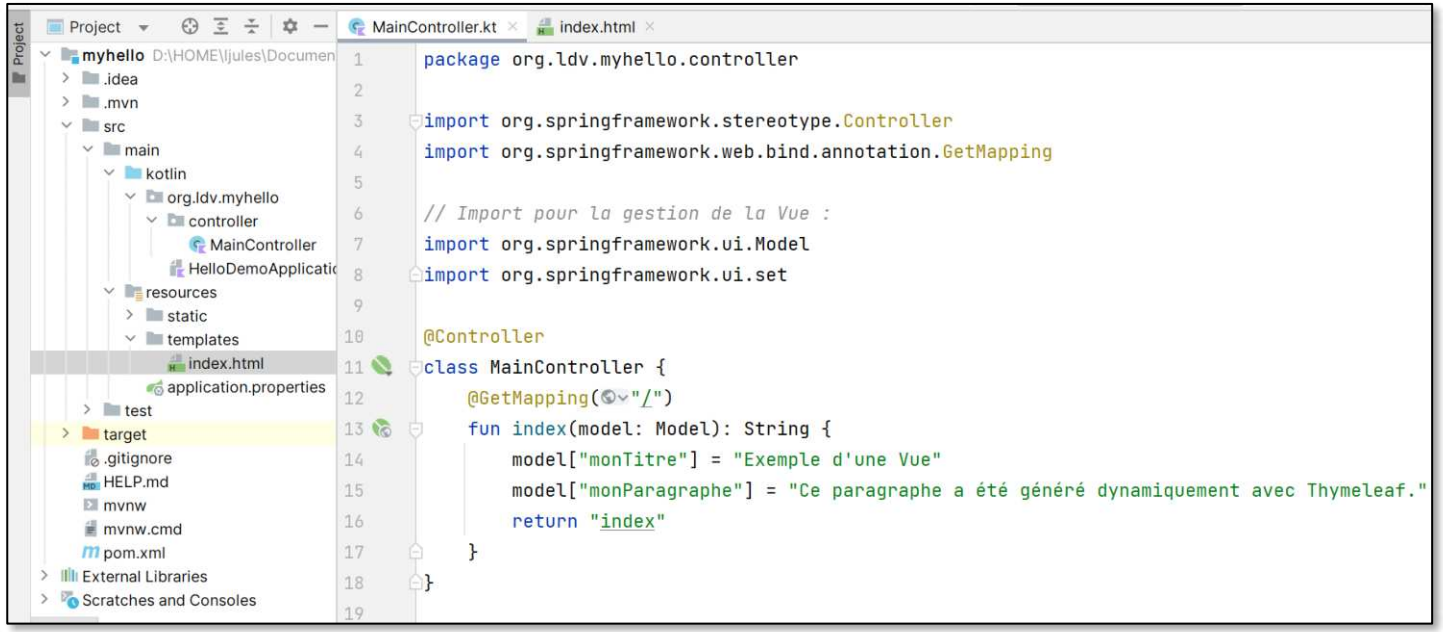
- **Génération de notre 1ère vue :**
  - La partie Contrôleur :
    - La méthode **`index()`** contient un paramètre de type **`Model`**, c'est une collection qui fonctionne comme un **`Map (clé/valeur)`** et qui sera envoyé à notre template pour résoudre les variables du template.

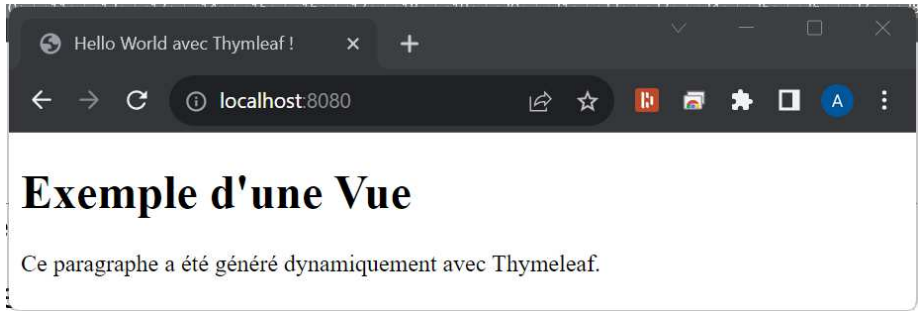
```
@Controller
class MainController {
    @GetMapping("/")
    fun index(model: Model): String {
        model["monTitre"] = "Exemple d'une Vue"
        model["monParagraphe"] = "Ce paragraphe a été généré dynamiquement avec Thymeleaf."
        return "index"
    }
}
```

Chemin relatif du fichier template à partir du dossier templates

La couche Vue

- **Génération de notre 1ère vue :**
  - Tester le bon fonctionnement : http://localhost:8080/



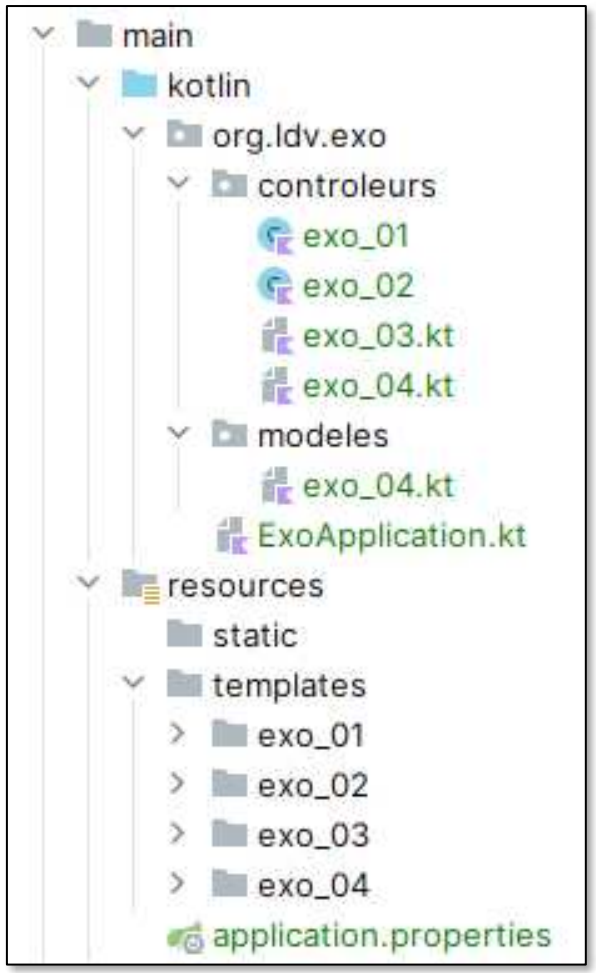


Exercices de renforcement

Organisation préalable

- **Organisation de votre structure de projet :**

  - Afin de gagner du temps nous allons réaliser tous les exercices dans le même projet Spring Boot.
  - On notera chaque fichier par le nom de l'exercice (ex : `exo_01.kt`)
  - On créera un dossier template pour chaque exercice pour y placer les Vues correspondantes.
  - Et globalement notre projet contiendra 2 dossiers :
    - **`controleurs`** : Pour recevoir tous les fichiers contrôleur de classe de chaque exercice
    - **`modeles`** : Pour recevoir le fichier comportant la classe métier des exercices



Exercice-01

- **Commande d'un fast-food :**
  - Un client passe une commande pour un site de livraison en ligne. La commande est passée au moyen d'une requête **GET**.
  - Réaliser **le Contrôleur** et la **Vue** permettant d'afficher :
    - Le nom du client avec un titre **<h1>**
    - Le contenu de la commande sous la forme d'une liste
    - Les tirets bas « _ » devront être remplacés par des espaces

  - **Requête de test :**
    http://localhost:8080/wonderfast/mycommand?name=John&surname=DOE&starter=salade&maincourse=vegan_burger&desert=sunday_vanille&drink=Pepsi

  - **Rendu attendu :**



Exercice-02

- **Calcul d'une moyenne pondérée :**
  - Un ensemble de notes accompagnées de leurs coefficients est communiqué par une requête **GET** avec une **QuerryString** fournissant dans la même valeur le coefficient suivi de la note séparés par un tiret bas. Le nom des paramètres correspond au nom de l'évaluation.
  - **Requête de test :** http://localhost:8080/transmission-resultats?Devoir_maison=2_12&QCM=1_18&Expose=2_9&DS=3_11
  - La vue doit afficher un tableau affichant le détail des notes (nom de l'évaluation, coefficient et la note.
  - La dernière ligne du tableau indique la moyenne
  - **Rendu :**

Relevé de notes :

Evaluation	Coef.	Notes
Devoir maison	2.0	12.0
QCM	1.0	18.0
Expose	2.0	9.0
DS	3.0	11.0
MOYENNE	11.625	

Exercice-03

- **Récupération d'un formulaire :**
  - Nous allons ici travailler avec un formulaire pour gérer une requête **POST**.

  - **<u>Travail préalable :</u>**
    - Créer un **`dossier`** **`exo_03`** dans le dossier
    - Dans le dossier, créer le fichier **`formulaire.html`**
    - Coller dans le fichier le code suivant :
    - Le code produira la page ci-dessous :




```
<!DOCTYPE html>
<html lang="fr" xmlns:th="http://www.thymeleaf.org">
<head>
   <meta charset="UTF-8">
   <title>Formulaire d'inscription</title>
</head>

<body>
<h2>FORMULAIRE D'INSCRIPTION</h2>
   <form th:action="@{/reponse-formulaire}" method="post">
      <label for="titre">Titre :</label>
      <select id="titre" name="titre">
         <option value="M.">M.</option>
         <option value="Mme">Mme</option>
      </select>
      <br>
      <label for="nom">Nom :</label>
      <input type="text" id="nom" name="nom" required>
      <br>
      <label for="prenom">Prénom :</label>
      <input type="text" id="prenom" name="prenom" required>
      <br>
      <label for="dateNaissance">Date de Naissance :</label>
      <input type="date" id="dateNaissance" name="dateNaissance" required>
      <br>
      <button type="submit">Envoyer</button>
   </form>
</body>
</html>
```

Exercice-03

- **Récupération d'un formulaire :**

  - **Travail préalable :**
    - Créer votre **Contrôleur** en créant le fichier `exo_03.kt`
    - Dans cette la classe `exo_03` nous auront 2 mappings de méthode :

      - 1 mapping pour le **endpoint** `"/exo_3/formulaire"`, ce mapping permet d'appeler votre formulaire.

      - 1 mapping pour le **endpoint** `"/reponse-formulaire"`, afin de récupérer les données du formulaire.

Exercice-03

- **Récupération d'un formulaire :**
  - **<u>Travail préalable :</u>**
    - On vous donne une 1ère version de votre Contrôleur ci-contre
    - Ce code sera à modifier selon les préconisations qui seront citées après.

```
@Controller
class exo03 {
    new *
    @GetMapping("/exo_03/formulaire")
    fun afficherFormulaire(): String {
        return "exo_03/formulaire"
    }
    new *
    @PostMapping("/reponse-formulaire")
    fun traitementFormulaire(
        @RequestParam("titre") titre: String,
        @RequestParam("nom") nom: String,
        @RequestParam("prenom") prenom: String,
        @RequestParam("dateNaissance") @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) dateNaissance: LocalDate,
        // A COMPLETER
    ): String {
        // Formatage de la date de naissance :
        val dateFormatee = dateNaissance.format(DateTimeFormatter.ofPattern( pattern: "dd-MM-yyyy"))

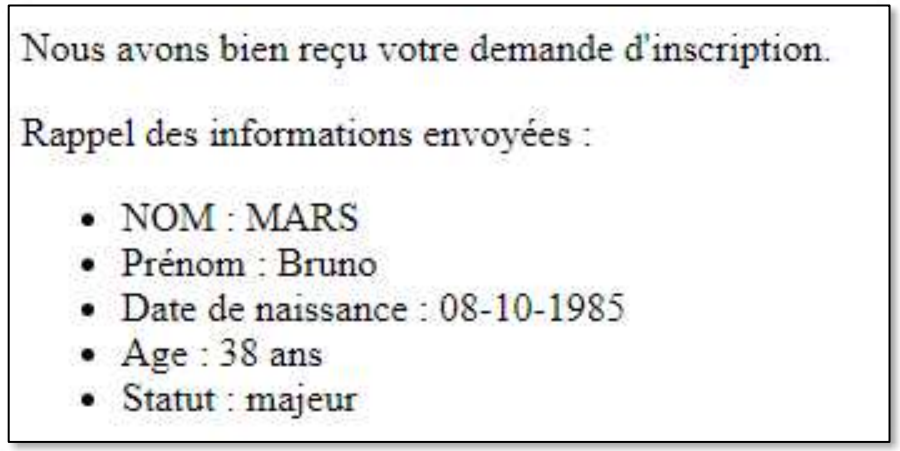
        // A COMPLETER
        return "exo_03/confirmationInscription"
    }
}
```

Exercice-03

- **Récupération d'un formulaire :**

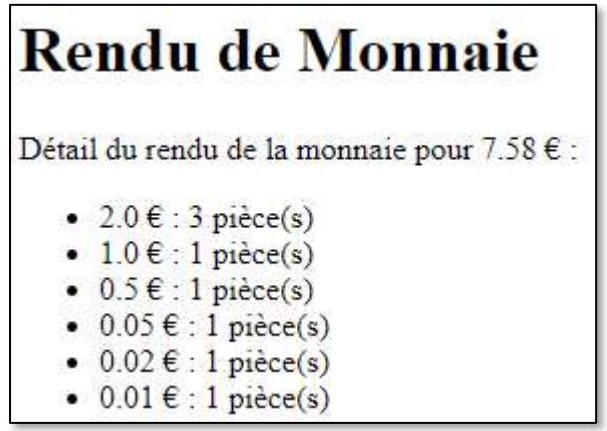
  - **<u>Travail à faire :</u>**
    - Créer la **Vue** affichant la page de confirmation de l'inscription.
    - Apporter les ajouts nécessaires au Contrôleur pour obtenir les informations nécessaires à la **Vue** de la page de confirmation.
    - Ci-dessous un exemple du résultat attendu :





Exercice-04

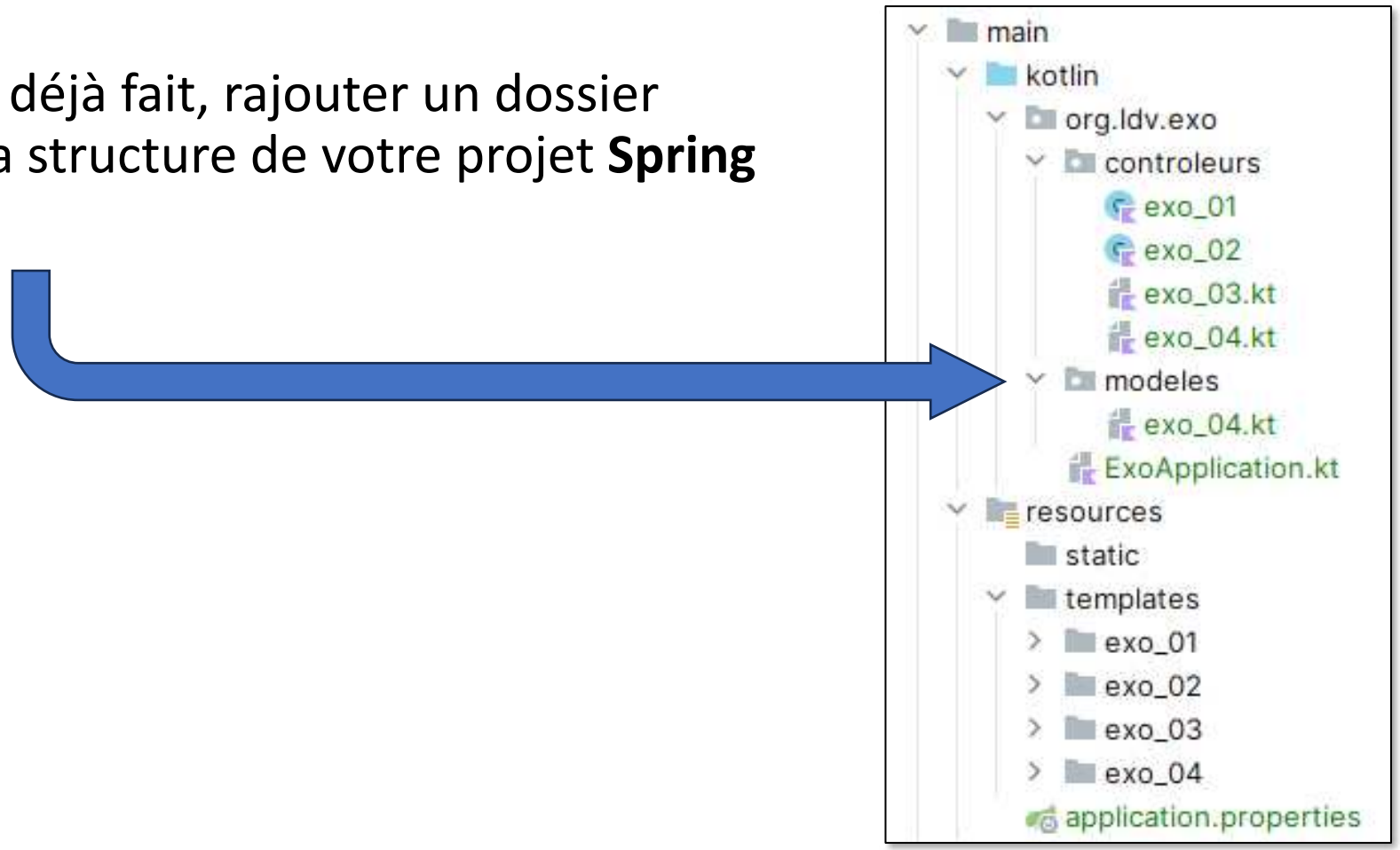
- **Un peu de logique métier :**
  - Jusqu'à présent nous nous sommes concentrés sur les relations entre la couche **Contrôleur** & **Vue**. Nous allons dans cet exercice introduire la notion de **logique métier** en intégrant la couche **Modèle.**
- **Exposé de la logique métier : Le rendu de monnaie :**
  - Notre application devra réaliser une tâche relativement simple :
    - Le Contrôleur reçoit une **URI** avec une **QuerryString** avec un paramètre (rendre) associé à une valeur qui est une somme en euros.
    - Exemple d'**URI** complète pour notre application :
    - http://localhost:8080/monnayer?rendre=7.58
  - La **Vue** affichera le détail du rendu de la manière suivante :



Exercice-04

- **Organisation du code :**

  - Si cela n'est pas déjà fait, rajouter un dossier **modeles** dans la structure de votre projet **Spring Boot** :



Exercice-04

- **Organisation du code :**
  - Pour vous aider à démarrer cet exercice, voici quelques éléments pour le Contrôleur et le Modèle (code partie à compléter). Vous êtes autonome pour la **Vue** car elle ne comporte pas de difficultés particulières par rapport aux exercices précédents.

  - **<u>Eléments de code pour le Contrôleur :</u>**

```
// Import des classes métier
import org.ldv.exo.modeles.Monnayer
import org.springframework.ui.Model
import org.springframework.ui.set
```

```
class MonnayerController {
    new *
    @GetMapping("/monnayer")
    fun rendreMonnaie(@RequestParam("rendre") somme: Double, model: Model): String {
        // Appel de la classe Monnayer pour effectuer le rendu de monnaie
        val renduMonnaie = Monnayer.rendre(somme)
```

Exercice-04

- **Organisation du code :**
  - Pour vous aider à démarrer cet exercice, voici quelques éléments pour le **Contrôleur** et le **Modèle** (code partie à compléter). Vous êtes autonome pour la **Vue** car elle ne comporte pas de difficultés particulières par rapport aux exercices précédents.

  - **<u>Eléments de code pour le Modèle :</u>**

```
package org.ldv.exo.modeles

new *
object Monnayer {
    new *
    fun rendre(somme: Double): Map<Double, Int> {
        val pieces = listOf(2.0, 1.0, 0.5, 0.2, 0.1, 0.05, 0.02, 0.01)
        val rendu = mutableMapOf<Double, Int>()
        var montantRestant = somme

        // A COMPLETER ....

        return rendu
    }
}
```

Exercice-04

- **Amélioration de la Vue :**
  - Notre **Vue** est un peu austère. Pour la rendre plus agréable, nous désirons utiliser des images pour chaque pièce.



  - On vous donne les images de chaque pièce.



Lire page suivante

Exercice-04

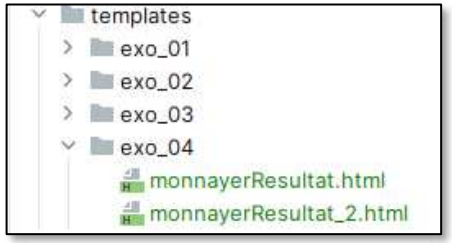
- **Amélioration de la Vue :**
  - N’effacez pas votre template précédent, mais créer un nouveau avec un nom différent, et vous renseignerez le nom du nouveau template dans votre **Vue**.

  - **Thymeleaf** n’est pas toujours évident à manipuler, on vous donne des éléments du code du template de la **Vue** :

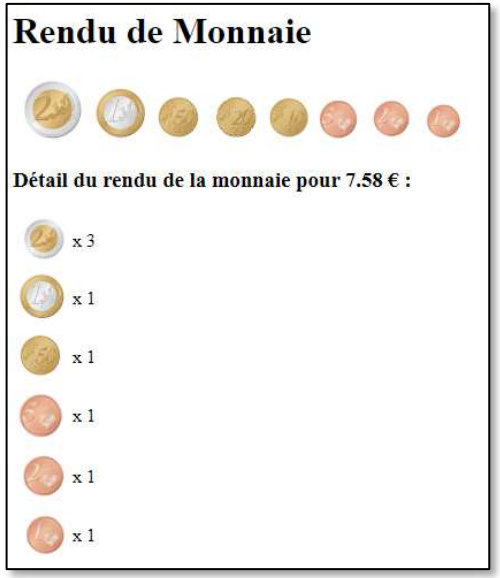
```
<body>
  <h1>Rendu de Monnaie</h1>
  <img src="/images/exo_04/piece-2_0-euro.png" width=65>
  <img src="/images/exo_04/piece-1_0-euro.png" width=50>
  <img src="/images/exo_04/piece-0_5-euro.png" width=45>
  <img src="/images/exo_04/piece-0_2-euro.png" width=43>
  <img src="/images/exo_04/piece-0_1-euro.png" width=41>
  <img src="/images/exo_04/piece-0_05-euro.png" width=40>
  <img src="/images/exo_04/piece-0_02-euro.png" width=40>
  <img src="/images/exo_04/piece-0_01-euro.png" width=40>

  <h3>Détail du rendu de la monnaie pour <span th:text="${somme}"></span> € :</h3>

  <table>
    <tr th:each="piece : ${renduMonnaie}">
      <td><img th:src="${'/images/exo_04/' + 'piece-' + #strings.replace(piece.key.toString(), '.', '_') + '-euro.png'}" alt="Piece" width=45
/></td>
       <td>x <span th:text="${piece.value}"> </span></td>
    </tr>
  </table>
</body>
```



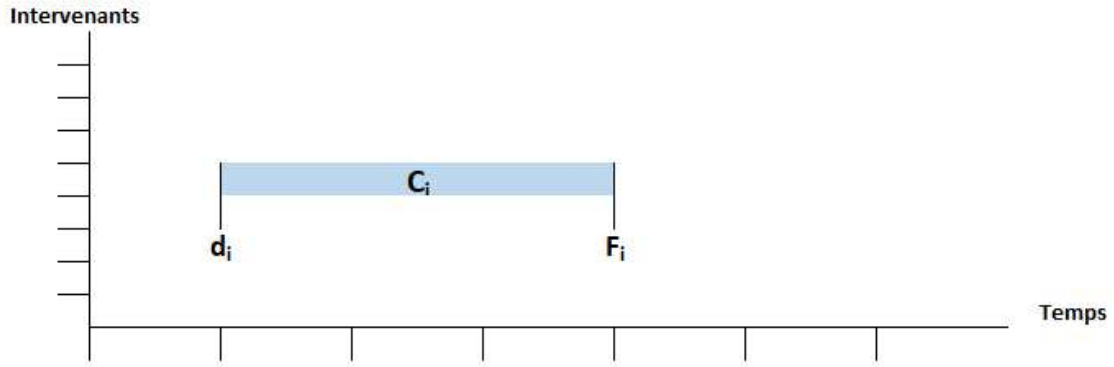
Proposition d’organisation des Templates.



Rendu attendu pour votre vue.

Exercice-05

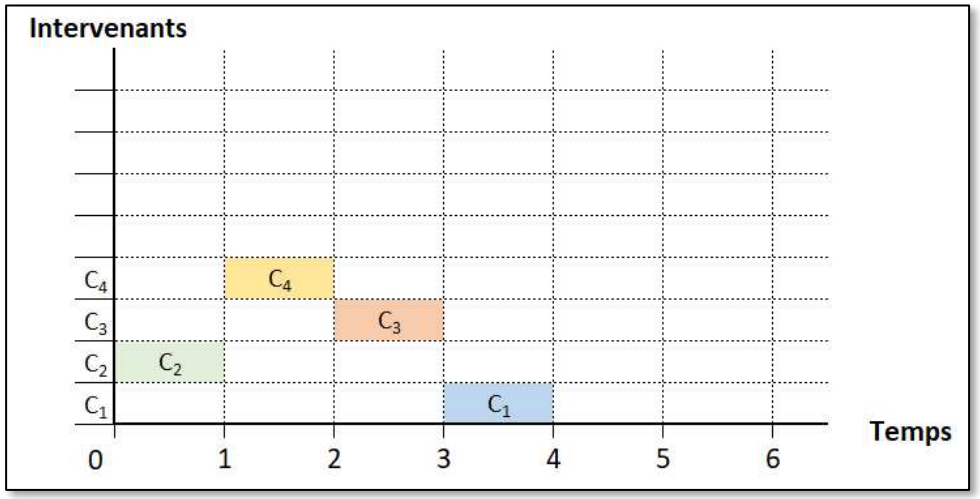
- **Planificateur de tâches :**
- **<u>Description de la problèmatique métier :</u>**
  - Des conférenciers sont invités à présenter leurs exposés dans une salle. Mais leurs disponibilités ne leur permettent d'intervenir qu'à des horaires bien définis. Le problème est de construire un planning d'occupation de la salle avec le plus grand nombre de conférenciers.
  - Désignons par **n**, entier naturel non nul, le nombre de conférenciers. Chacun d'eux, identifié par une lettre C_i , où **i** est un entier compris entre **0** et **n – 1**, est associé à un intervalle temporel $[d_i , f_i]$ où d_i et f_i désignent respectivement l'heure de début et l'heure de fin de l'intervention. Afin de dégager une tactique de résolution du problème, commençons par analyser plusieurs situations.



Exercice-05

- **Planificateur de tâches :**
- **<u>Illustration :</u>**

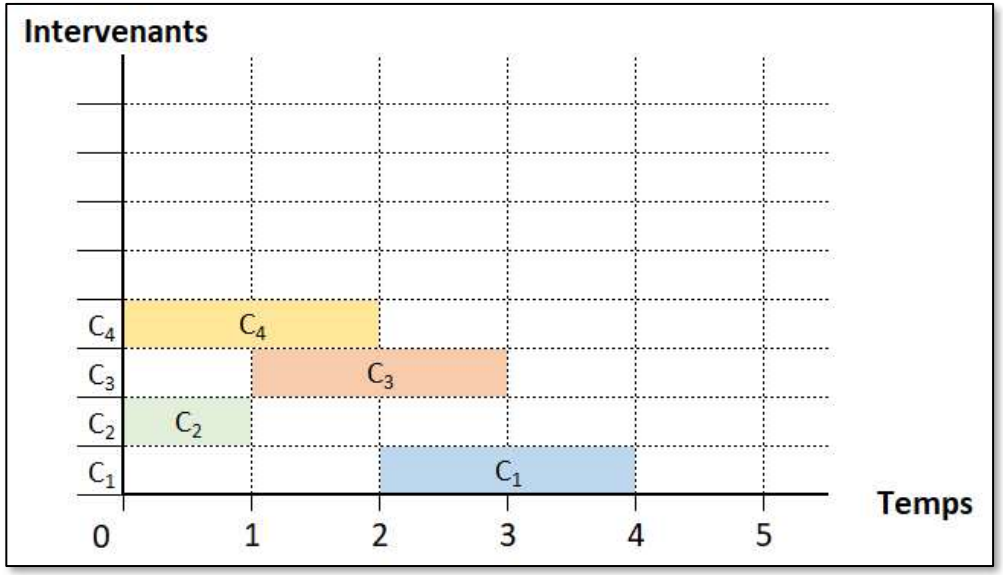
- **<u>Etude de cas 1 :</u>**
  - 4 conférenciers peuvent intervenir aux intervalles temporels suivants :
    - C_1 : [3, 4]
    - C_2 : [0, 1]
    - C_3 : [2, 3]
    - C_4 : [1, 2]
  - Une telle situation est simple puisque tous les conférenciers peuvent intervenir sur des créneaux horaires disjoints.
  - Le planning est donc défini par la suite **[C_2, C_4, C_3, C_1]** de conférenciers.



Exercice-05

- **Planificateur de tâches :**
- **Illustration :**

- **Etude de cas 2 :**
  - On reprend 4 conférenciers mais avec des créneaux pas toujours disjoints :
    - C_1 : [2, 4]
    - C_2 : [0, 1]
    - C_3 : [1, 3]
    - C_4 : [0, 2]
  - Ici **il faut faire des choix** et notamment ne pas retenir toutes les conférences. Les solutions possibles sont :
  - **[C_2, C_1] ou [C_2, C_3] ou [C_4, C_1] ou [C_3]**



La stratégie d'un algorithme glouton va nous permettre d'appliquer un choix !

Exercice-05

- **Proposition & description d'un algorithme selon une stratégie glouton :**

  - Classer les intervalles par heures de fin f_i croissantes ;
  - Choisir le conférencier C_i associer au 1er intervalle issu du tri croissant selon f_i
  - Choix parmi les intervalles suivant du classement le 1er qui est compatible avec le 1er conférencier;
  - Recommencer avec les intervalles classés suivant jusqu'à que l'ensemble des intervalles soient traités.

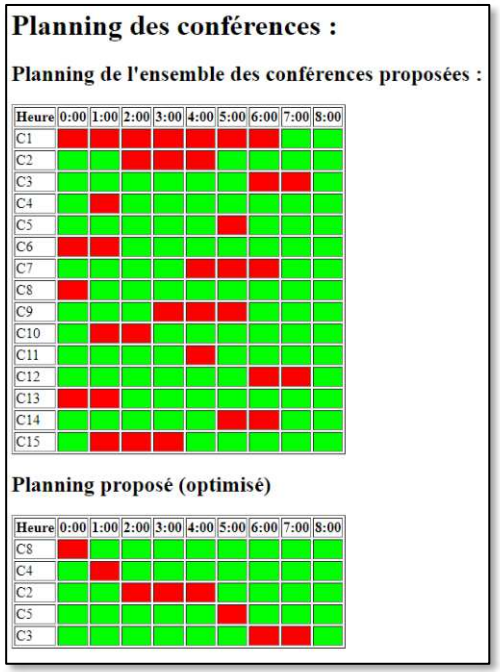
  - Nous vous fournissons le code de la logique métier, votre travail pour cet exercice se concentrera sur la bonne articulation des différents composants nécessaire, à savoir :
    - Un **Contrôleur** qui répondra à une requête contenant une QuerryString
    - Un **Modèle** composé de 2 classes :
      - Une classe **`Organizer`** qui instancie l'organisation d'une session de conférences.
      - Une classe **`Creneau`** qui modélise le créneau horaire occupé par une conférence (désignation, heure de début et heure de fin).
    - Une **Vue** pour présenter le planning de l'ensemble des conférences et un planning optimisé de choix de conférences.
    - Un **Service** : C'est la nouveauté de cet exercice, notre service est ici un composant qui va assurer l'instanciation des objets **`Organizer`**.

Exercice-05

- **Rendu attendu :**

  - Une fois votre application opérationnelle, vous pourrez la tester avec l'**URL** suivante :
    - http://localhost:8080/organizer?C1=0_7&C2=2_5&C3=6_8&C4=1_2&C5=5_6&C6=0_2&C7=4_7&C8=0_1&C9=3_6&C10=1_3&C11=4_5&C12=6_8&C13=0_2&C14=5_7&C15=1_4

  - **Vue attendue :**
    - L'URL fourni doit vous donner la **Vue** suivante :



Planning des conférences :

Planning de l'ensemble des conférences proposées :

Heure	0:00	1:00	2:00	3:00	4:00	5:00	6:00	7:00	8:00
C1									
C2									
C3									
C4									
C5									
C6									
C7									
C8									
C9									
C10									
C11									
C12									
C13									
C14									
C15									

Planning proposé (optimisé)

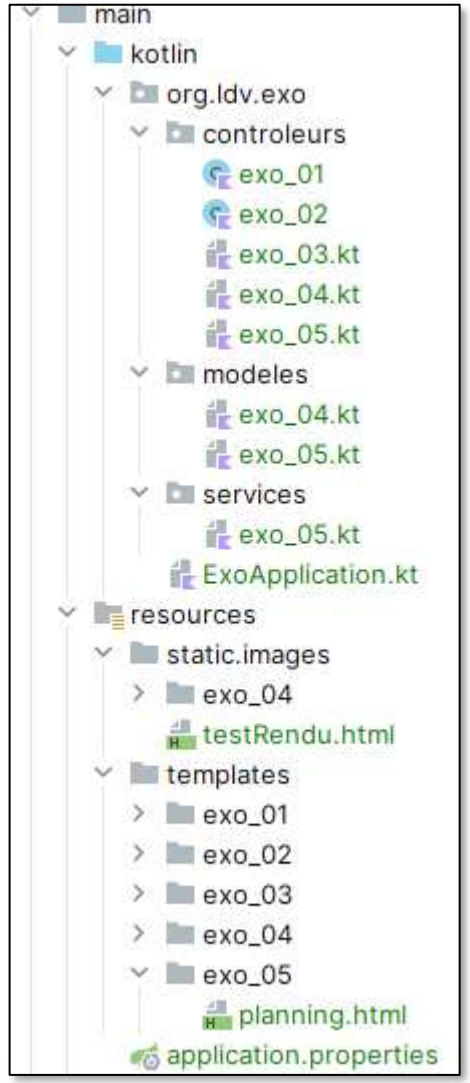
Heure	0:00	1:00	2:00	3:00	4:00	5:00	6:00	7:00	8:00
C8									
C4									
C2									
C5									
C3									

Lire page suivante

Exercice-05

- **Rendu attendu :**

  - **<u>Travail à réaliser :</u>**
    - Organisation de vos fichiers :
      - Nous utilisons le même schéma que les exercices précédents, mais nous rajoutons ici un nouveau dossier/package pour les services :



Lire page suivante

Exercice-05

- **Rendu attendu :**

  - **<u>Travail à réaliser :</u>**
    - Réalisation du modèle :
      - Déposer dans le bon fichier le code du modèle de notre application. Ce code comporte les 2 classes : **`Creneau`** et **`Organizer`**

Code sur la page suivante

Exercice-05

```
package org.ldv.exo.modeles

/**
 * Classe représentant l'organisation d'une journée de conférence.
 */
class Organizer(val listeCreneaux: List<Creneau>) {
    /**
     * Applique un algorithme glouton pour sélectionner et organiser
     * les créneaux.
     * @param creneaux Une liste contenant des objets Creneau.
     * @return Une liste contenant les créneaux sélection et ordonnés.
     */
    fun organize(): List<Creneau> {
        val planning = mutableListOf<Creneau>()

        // Tri des créneaux dans l'ordre des dates de fin croissantes :
        val sortedCreneaux = listeCreneaux.sortedBy { it.heureFin }

        // Construction du planning selon une sélection gloutonne :
        var i = 0
        var j = i + 1
        planning.add(sortedCreneaux[i])

        while (j < sortedCreneaux.size) {
            // Vérification de la compatibilité des intervalles :
            if (sortedCreneaux[i].heureFin <= sortedCreneaux[j].heureDebut) {
                planning.add(sortedCreneaux[j])
                i = j // On poursuit le travail sur le nouvel intervalle ajouté au planning.
            } else {
                j++
            }
        }
        return planning
    }

    /**
     * Affiche dans la console le planning donné en argument.
     */
    fun show(listePlanning: List<Creneau>) {
        var t = ""
        for (creneau in listePlanning) {
            t += "${creneau.nomConf}\t:${" ".repeat(creneau.heureDebut)}${"#".repeat(creneau.heureFin -creneau.heureDebut)}\n"
        }
        println(t)
    }

    /**
     * Affiche tous les créneaux proposés initialement.
     */
    fun showAll() {
        show(listeCreneaux)
    }

    fun showOptimised() {
        show(organize())
    }

}
```

A placer à la suite du code précédent

```
/**
 * Modélise un créneau de cours/conférence avec 3 attributs :
 * @property nomConf Désignation du cours/conférence.
 * @property heureDebut Heure du début du cours/conférence.
 * @property heureFin Heure de fin du cours/conférence.
 */
class Creneau(val nomConf: String, val heureDebut: Int, val heureFin: Int) {
    fun duree() = heureFin - heureDebut
    override fun toString(): String {
        return "[${nomConf} : ${heureDebut} -> ${heureFin}]"
    }
}
```

Page suivante pour le Service

Exercice-05

- **Le service :**
  - C'est la nouveauté de cet exercice, c'est une simple classe qui possède une méthode pour instancier des objets de la classe **`Organizer`**. Elle est annotée par **`@Service`** :

```
package org.ldv.exo.services

import org.springframework.stereotype.Service

import org.ldv.exo.modeles.Organizer
import org.ldv.exo.modeles.Creneau

/**
 * Service qui prend en charge les instances de la classe Organizer.
 */
@Service
class OrganizerService {
    fun createOrganizer(listeCreneaux: List<Creneau>): Organizer {
        return Organizer(listeCreneaux)
    }
}
```

Page suivante pour la Vue

Exercice-05

- **La Vue**

  - La vue est ici assez complexe, elle va recevoir de la part du **Contrôleur** :

    - Une liste **`creneauxComplet`** qui comporte des objets **`Creneaux`** correspondants à l'ensemble des créneaux fournis par l'**URL**.

    - Une liste **`creneauxOptimised`** qui comporte les **`Creneaux`** effectivement conservés et plannifiés.

    - La Vue reçoit également une valeur **`maxTime`** qui correspond au temps occupé par le planning transmis.

  - Aller sur la page suivante et recopier le code de la vue dans le fichier adéquate.

Exercice-05

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8">
  <title>Votre Page</title>
  <style>
    .occupe {
      background-color: #FF0000; /* Changez la couleur selon vos préférences */
    }
    .libre {
      background-color: #00FF00; /* Changez la couleur selon vos préférences */
    }
  </style>
</head>
<body>
  <h1>Planning des conférences :</h1>
  <h2>Planning de l'ensemble des conférences proposées :</h2>
  <table border="1">
    <tr>
      <th>Heure</th>
      <th th:each="heure : ${#numbers.sequence(0, maxTime)}" th:text="${heure + ':00'}"></th>
    </tr>
    <tr th:each="creneau : ${creneauxComplet}">
      <td th:text="${creneau.nomConf}"></td>
      <td th:each="heure : ${#numbers.sequence(0, maxTime)}"
        th:classappend="${creneau.heureDebut <= heure and heure < creneau.heureFin} ? 'occupe' : 'libre'">
      </td>
    </tr>
  </table>

  <h2>Planning proposé (optimisé)</h2>
  <table border="1">
    <tr>
      <th>Heure</th>
      <th th:each="heure : ${#numbers.sequence(0, maxTime)}" th:text="${heure + ':00'}"></th>
    </tr>
    <tr th:each="creneau : ${creneauxOptimised}">
      <td th:text="${creneau.nomConf}"></td>
      <td th:each="heure : ${#numbers.sequence(0, maxTime)}"
        th:classappend="${creneau.heureDebut <= heure and heure < creneau.heureFin} ? 'occupe' : 'libre'">
      </td>
    </tr>
  </table>
</body>
</html>
```

Exercice-05

- **Le Contrôleur :**
  - Nous vous donnons qu'une partie du code du Contrôleur. La partie manquante correspond à des acquis des exercices précédents.
  - Vous observerez l'apparition du service

Code sur la page suivante

Exercice-05

```
@Controller
class OrganizerController(val organizerService: OrganizerService) {
    new *
    @GetMapping("/organizer")
    fun mapOrganizer(@RequestParam querryMap: Map<String, String>, model: Model): String {

        // Extraction des données de l'URL :
        val listeCreneaux = mutableListOf<Creneau>()

        // Création de l'objet métier Organizer par son service :
        val organizer = organizerService.createOrganizer(listeCreneaux)

        // Mapping de la Vue :
        model["creneauxComplet"] = listeCreneaux
        model["creneauxOptimised"] = organizer.organize()
        val maxTime = listeCreneaux.map { it.heureFin }.maxOrNull() ?: 0
        model["maxTime"] = maxTime

        return "/exo_05/planning"
    }
}
```

A compléter…

Exercice-05

- **Tester votre application avec l'URL :**
  - http://localhost:8080/organizer?C1=0_7&C2=2_5&C3=6_8&C4=1_2&C5=5_6&C6=0_2&C7=4_7&C8=0_1&C9=3_6&C10=1_3&C11=4_5&C12=6_8&C13=0_2&C14=5_7&C15=1_4

Fin